DATA 260: Agentic AI

Instructor: Prof. Dr. Simon Shim

Student Name: Trapti Kulshrestha

STUDENT ID: 019124833

Homework: 5

Section 0: Personal Configuration:

SID4	4833
PORT_BASE:	8333
PREFIX	s4833
SEED	4833
VERIFY_SEED	264833
DOMAIN_ID:	1
Assigned Domain	Clinical Trial Listings

Hardware: Apple MacBook Air M4

Local Model: qwen3:8b

Tagged Commit Hash:

GitHub Repository: https://github.com/Psycoder0611/data260-4833

Project Setup:

For Homework 5, I continued working in my existing DATA 260 repository and created the hw5-work branch to keep the new implementation separate from the previously submitted HW4 state.

I extended the existing FastAPI, MySQL, and React application rather than creating a separate project. For HW5, I added the Research Organization entity and its relationship with Clinical Trials, Redux Toolkit state management, two MCP servers, retry and fault-injection experiments, the unified execute_tool() layer, the local Ollama agent loop, offline tests, and final verification.

I also created the reports/hw05/ directory and its raw/ subdirectory to store the report, metrics, AI-use documentation, reflection, run logs, MCP outputs, agent logs, fault-injection results, and verification evidence required for submission.

Part 1 – Related Entity, CRUD, and Redux

1.1 Database Design and Related Entity:

I extended the existing Clinical Trial Listings database by adding a second entity called research_organizations. A research organization can be associated with multiple clinical trials, creating a one-to-many relationship.

The organization table contains a name, country, unique contact email, and timestamps. I also extended clinical_trials with a unique trial_code, numeric participant_capacity, organization_id foreign key, and timestamps. I used ON DELETE RESTRICT so an organization cannot be deleted while clinical trials are still associated with it.

```python
# ------------------------------------------------------------------
# HW5 Related Entity
# ------------------------------------------------------------------

class ResearchOrganizationDB(Base):
    __tablename__ = "research_organizations"

    id = Column(
        Integer,
        primary_key=True,
        index=True,
        autoincrement=True
    )

    name = Column(
        String(255),
        nullable=False
    )

    country = Column(
        String(100),
        nullable=False
    )
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                mysql

mysql> DESCRIBE research_organizations;
+---------------+--------------+------+-----+---------+----------------+
| Field         | Type         | Null | Key | Default | Extra          |
+---------------+--------------+------+-----+---------+----------------+
| id            | int          | NO   | PRI | NULL    | auto_increment |
| name          | varchar(255) | NO   |     | NULL    |                |
| country       | varchar(100) | NO   |     | NULL    |                |
| contact_email | varchar(255) | NO   | UNI | NULL    |                |
| created_at    | datetime     | NO   |     | NULL    |                |
| updated_at    | datetime     | NO   |     | NULL    |                |
+---------------+--------------+------+-----+---------+----------------+
6 rows in set (0.001 sec)

mysql> DESCRIBE clinical_trials;
+----------------------+--------------+------+-----+-------------------+-----------------------------------------------+
| Field                | Type         | Null | Key | Default           | Extra                                         |
+----------------------+--------------+------+-----+-------------------+-----------------------------------------------+
| id                   | int          | NO   | PRI | NULL              | auto_increment                                |
| trialTitle           | varchar(255) | NO   |     | NULL              |                                               |
| sponsor              | varchar(255) | NO   | MUL | NULL              |                                               |
| submitterEmail       | varchar(255) | YES  |     | NULL              |                                               |
| description          | text         | YES  |     | NULL              |                                               |
| category             | varchar(100) | YES  |     | NULL              |                                               |
| trial_code           | varchar(100) | YES  | UNI | NULL              |                                               |
| participant_capacity | int          | NO   |     | 100               |                                               |
| organization_id      | int          | YES  | MUL | NULL              |                                               |
| created_at           | datetime     | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED                             |
| updated_at           | datetime     | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+----------------------+--------------+------+-----+-------------------+-----------------------------------------------+
11 rows in set (0.001 sec)

mysql>
```

1.2 Pydantic Schemas:

I implemented the related-entity API using FastAPI and Pydantic. The create endpoint accepts the organization name, country, and contact email. The contact email is validated using Pydantic EmailStr, and MySQL enforces uniqueness. A successful request creates the organization and returns HTTP 201.

```python
class OrganizationCreate(BaseModel):
    name: str = Field(min_length=2, max_length=255)
    country: str = Field(min_length=2, max_length=100)
    contact_email: EmailStr


class OrganizationUpdate(BaseModel):
    name: str = Field(min_length=2, max_length=255)
    country: str = Field(min_length=2, max_length=100)
    contact_email: EmailStr

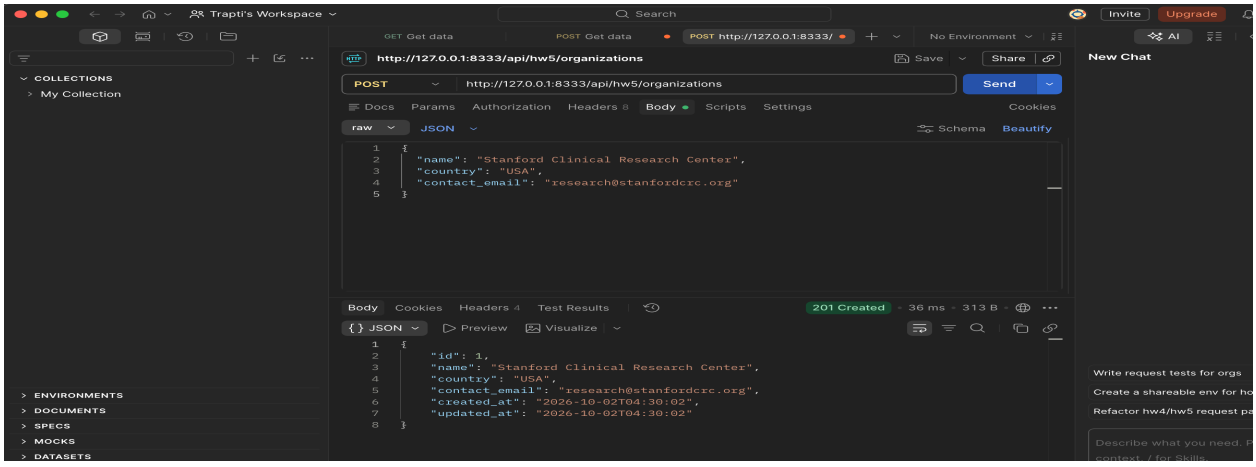

class OrganizationResponse(BaseModel):
    id: int
    name: str
    country: str
    contact_email: str
    created_at: datetime
    updated_at: datetime

    model_config = {
        "from_attributes": True
```

1.3 Research Organization CRUD:

1.3.1 Create Organization:

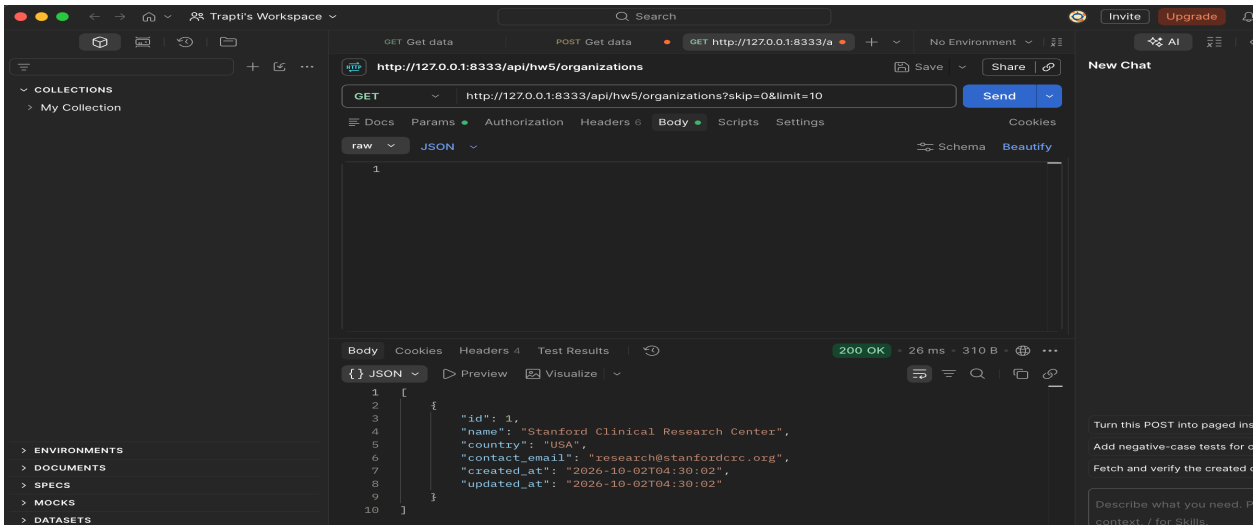
I implemented the POST endpoint to create a new research organization. A successful request stores the record in MySQL and returns HTTP 201.



1.3.2 List Organizations with Pagination:

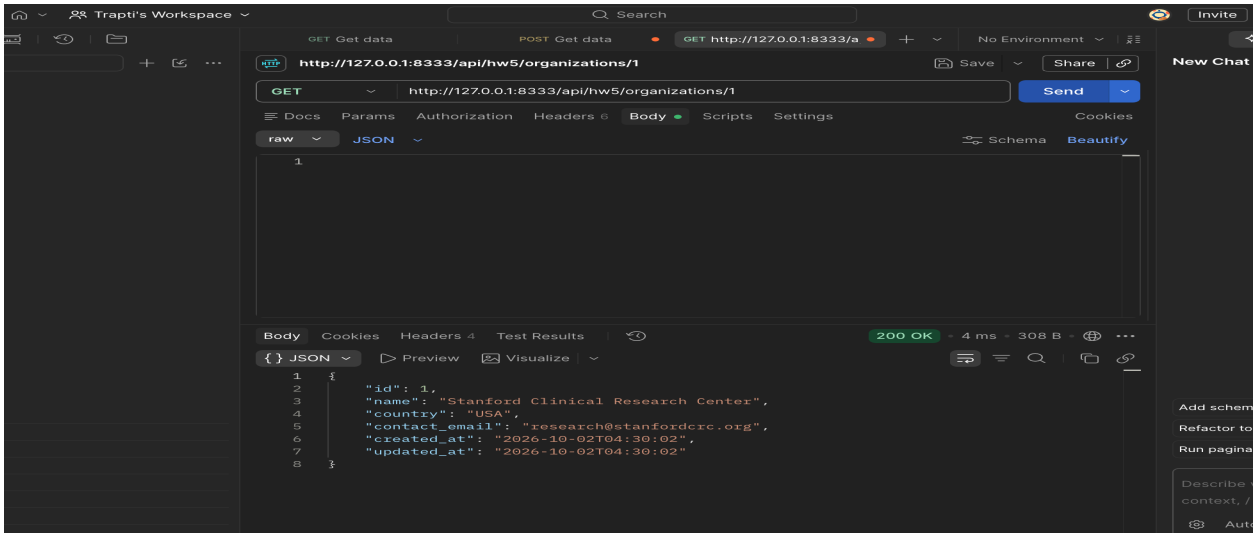
I implemented pagination using skip and limit. skip determines how many records are skipped, while limit controls the maximum number of records returned.

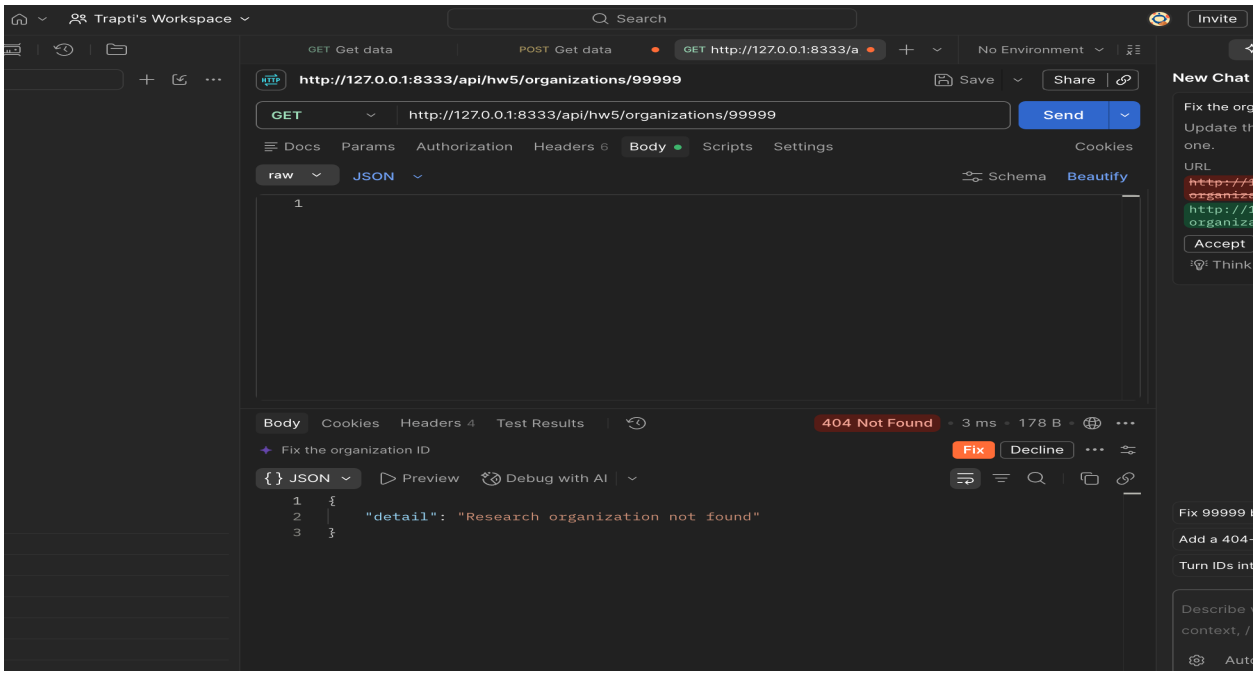
```python
def list_organizations(
    skip: int = Query(default=0, ge=0),
    limit: int = Query(default=10, ge=1, le=100),
    db: Session = Depends(get_db)
):
    return (
        db.query(ResearchOrganizationDB)
        .order_by(ResearchOrganizationDB.id)
        .offset(skip)
        .limit(limit)
        .all()
    )
```



1.3.3 Get Organization by ID:

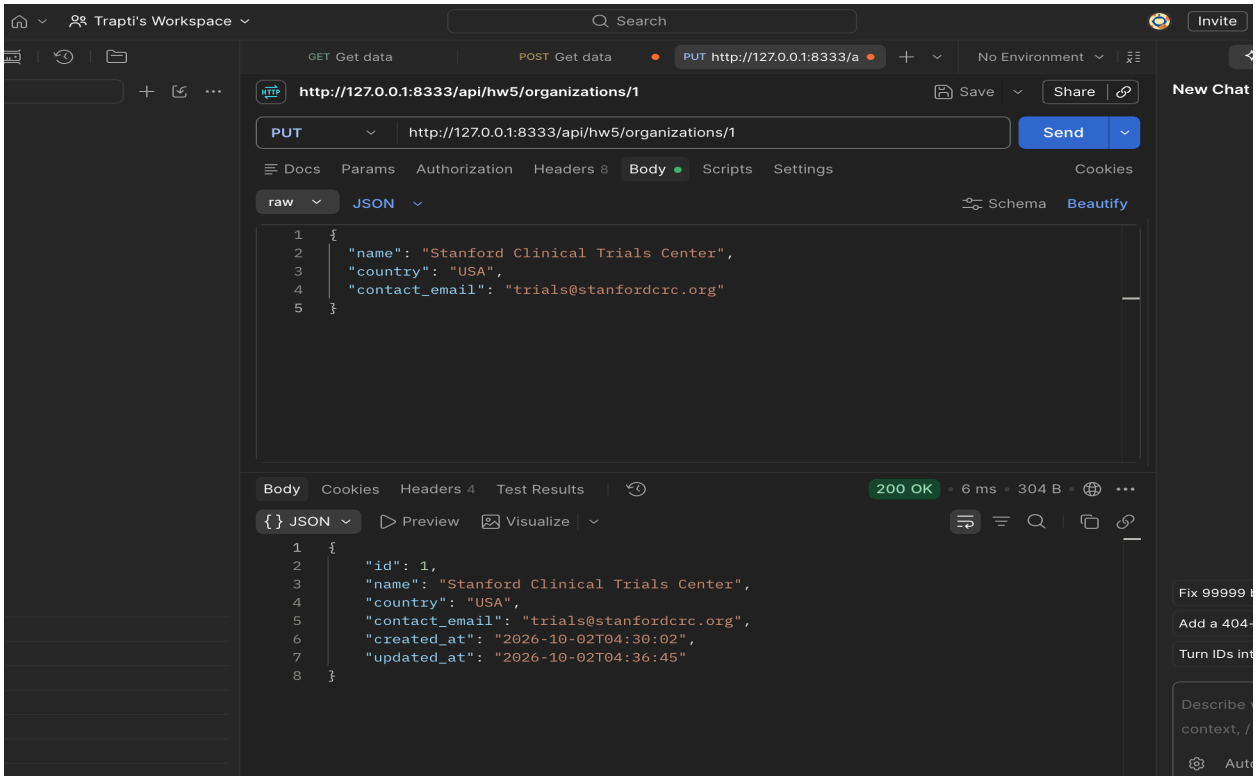
The GET-by-ID endpoint returns a research organization when a valid ID is provided. I also tested a non-existent organization ID to verify error handling. The API correctly returned HTTP 404 with the message Research organization not found.





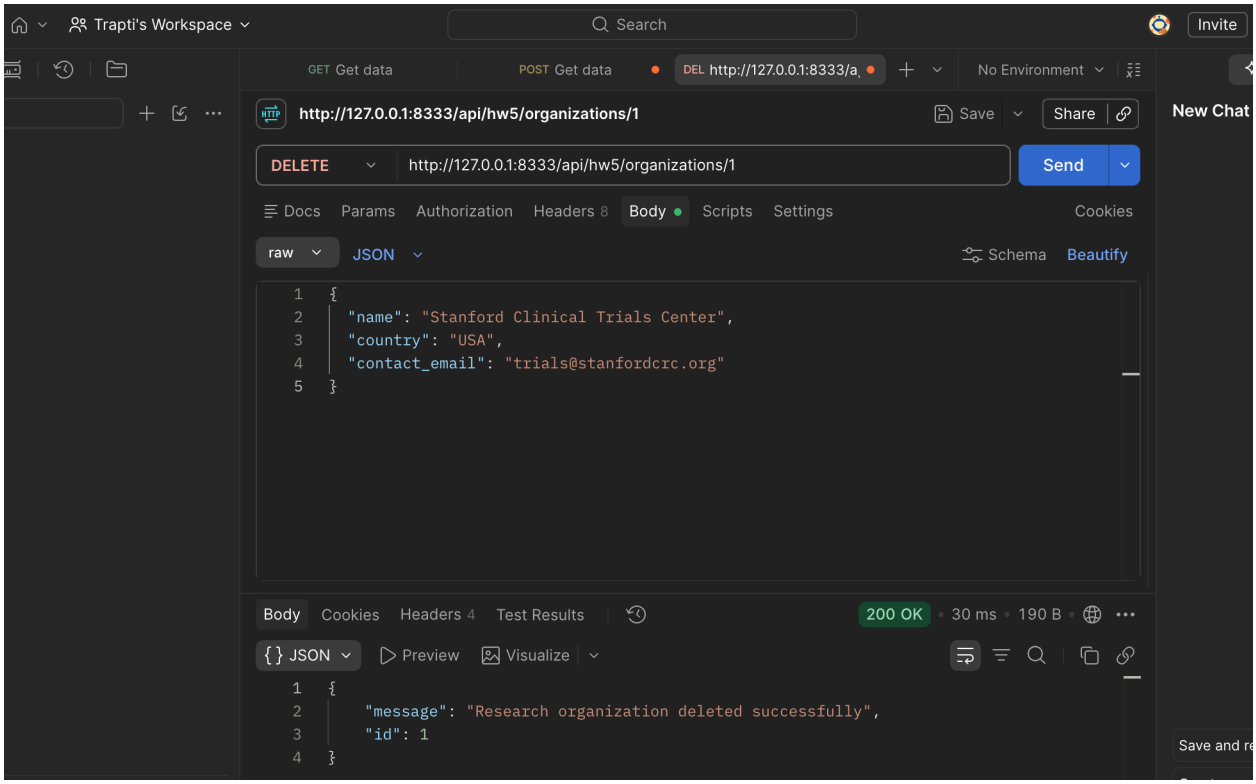
1.3.4 Update Organization:

I implemented the PUT endpoint to update the organization name, country, and contact email. The updated values are persisted in MySQL.



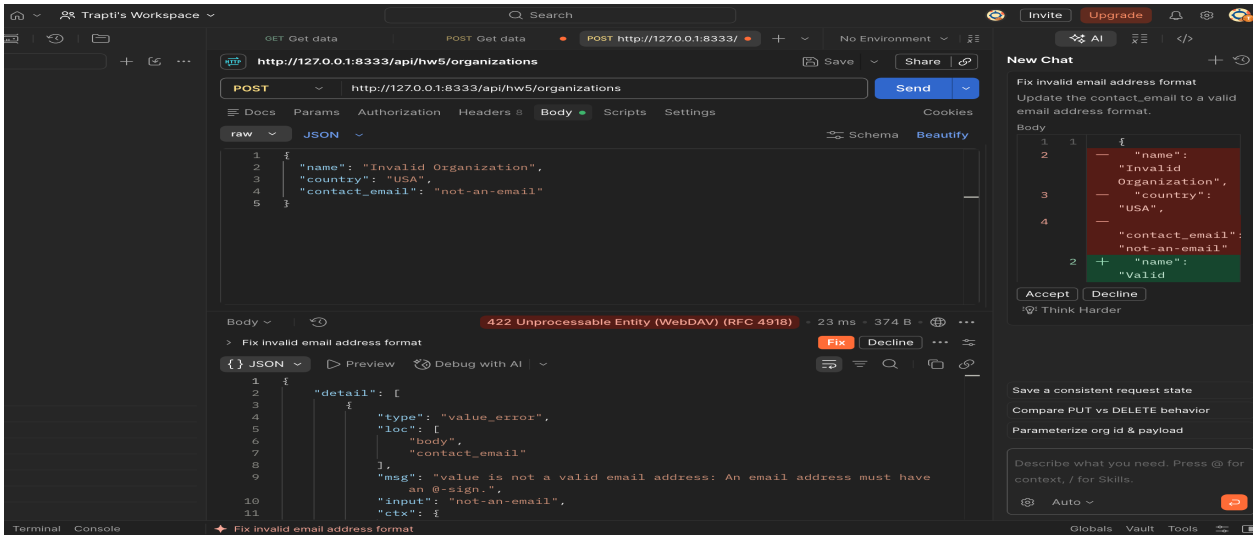
1.3.5 Delete Organization:

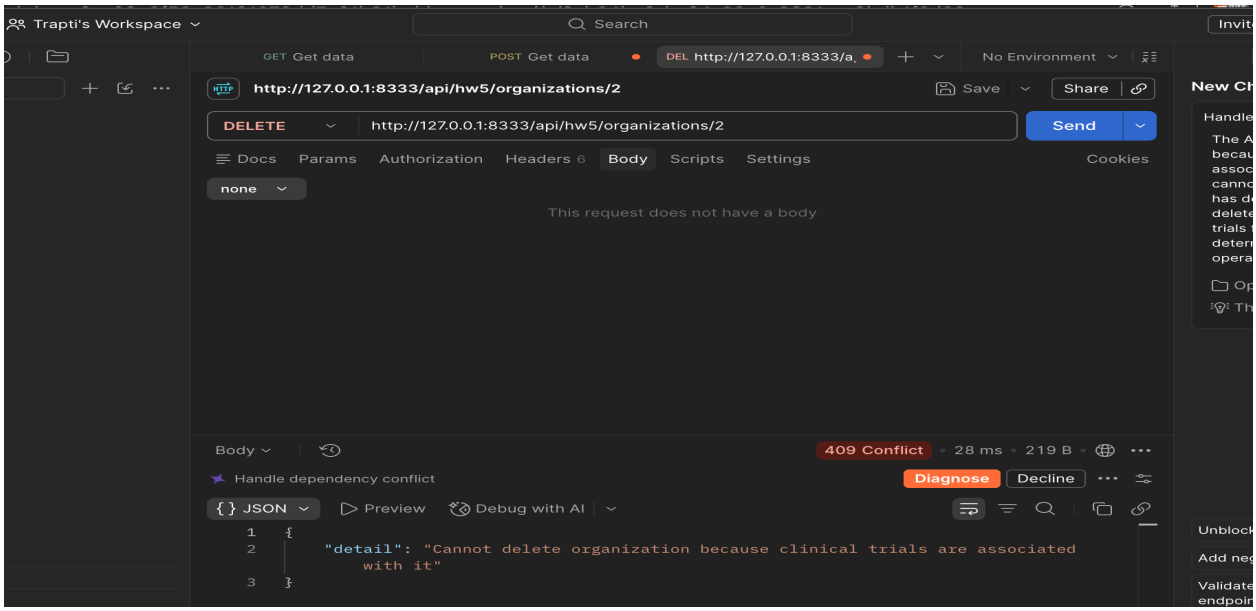
The DELETE endpoint removes an organization only when it is safe to do so. Later, I also test the relationship constraint to ensure an organization with associated clinical trials cannot be deleted.



1.3.6 Validation and Constraint Handling:

I tested an invalid contact email to verify input validation. Pydantic rejected the request before database insertion and returned HTTP 422 because the value was not a valid email address.





I also tested the relationship constraint by attempting to delete a research organization that still had a clinical trial associated with it. The API correctly blocked the deletion and returned HTTP 409, preventing the relationship from becoming invalid.

1.4 Clinical Trial CRUD:

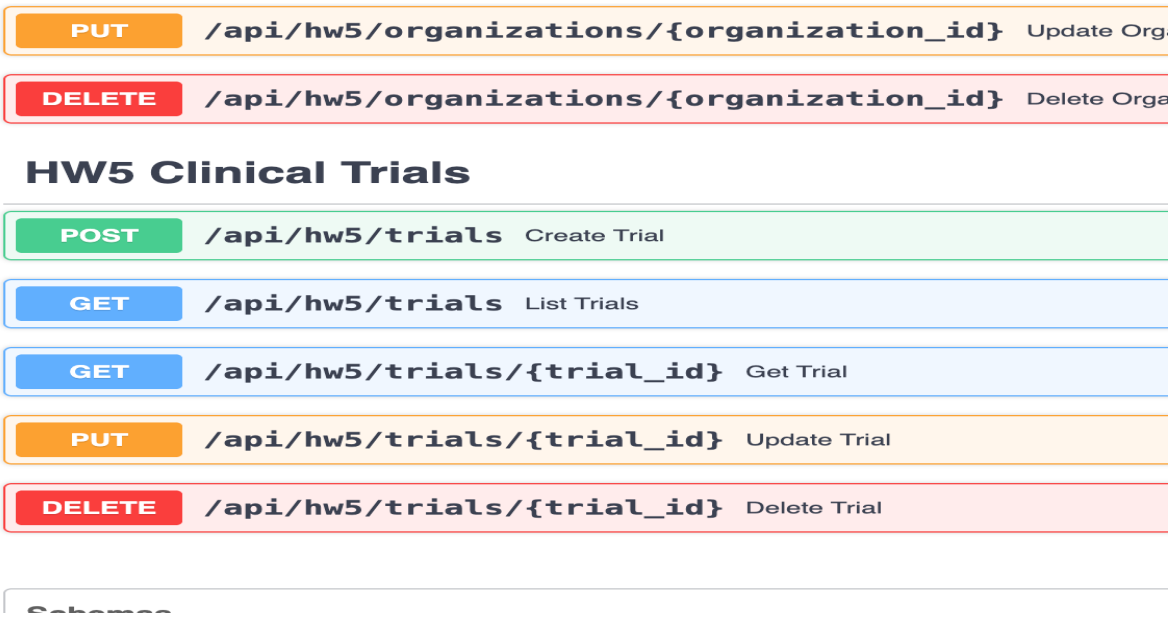
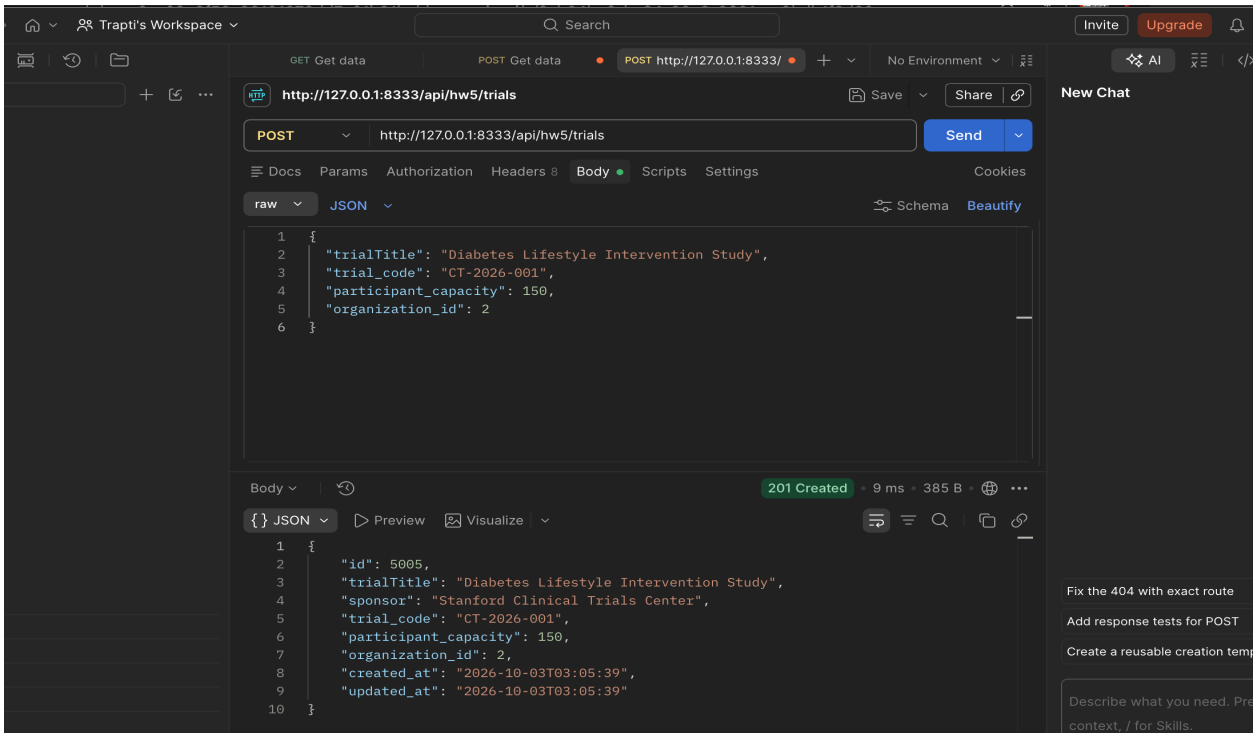


Figure: HW5 Clinical Trial CRUD endpoints registered in FastAPI Swagger.

1.4.1 Create Clinical Trial:

```python
def create_trial(
    trial: HW5TrialCreate,
    db: Session = Depends(get_db)
):
    organization = db.get(
        ResearchOrganizationDB,
        trial.organization_id
    )

    if organization is None:
        raise HTTPException(
            status_code=404,
            detail="Research organization not found"
        )
```

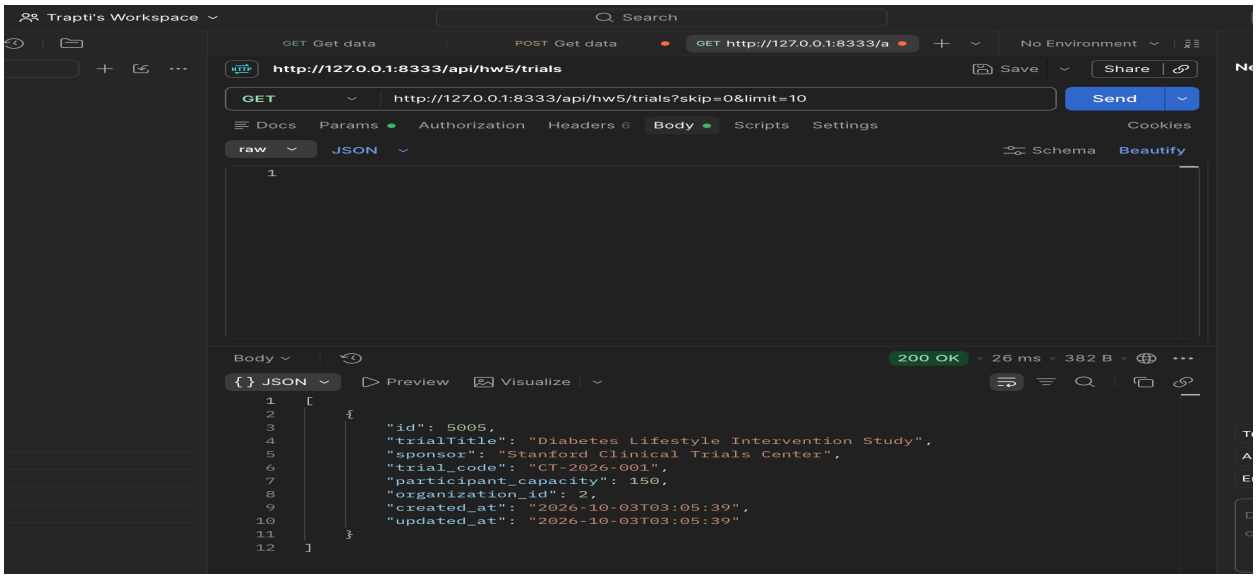


I implemented the POST endpoint to create a clinical trial linked to an existing research organization. Before inserting the record, the endpoint verifies that the organization exists. The trial_code is unique, participant_capacity is validated as a positive number, and the selected organization is stored through organization_id.

1.4.2 List Clinical Trials:

I implemented a paginated GET endpoint for the HW5 clinical-trial records. The endpoint uses skip and limit and returns the trials in ID order.

```python
def list_trials(
    skip: int = Query(default=0, ge=0),
    limit: int = Query(default=10, ge=1, le=100),
    db: Session = Depends(get_db)
):
    return (
        db.query(ClinicalTrialDB)
        .filter(
            ClinicalTrialDB.trial_code.isnot(None)
        )
        .order_by(ClinicalTrialDB.id)
        .offset(skip)
        .limit(limit)
        .all()
    )
```



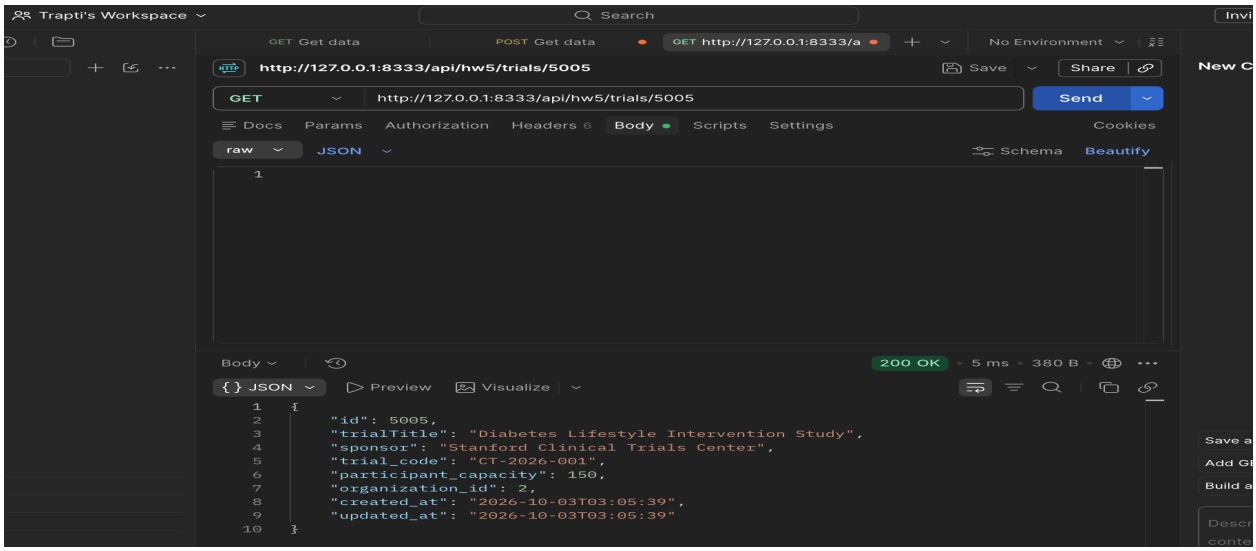
1.4.3 Get Clinical Trial by ID:

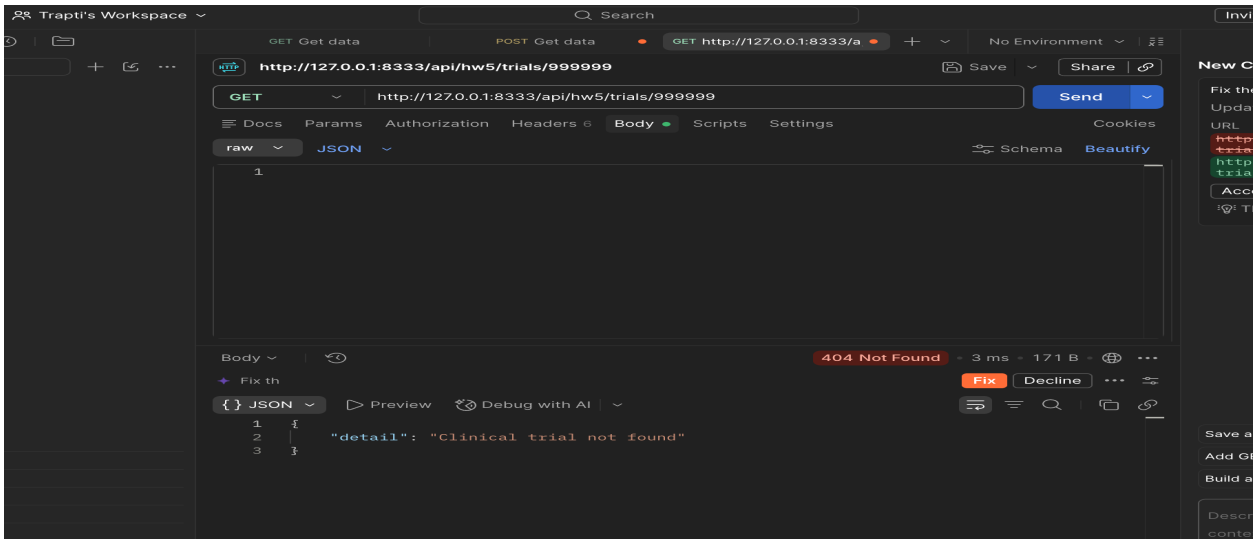
The GET-by-ID endpoint retrieves one clinical trial. I also tested a non-existent ID and verified that the API returns HTTP 404 with a clear error message.

```python
def get_trial(
    trial_id: int,
    db: Session = Depends(get_db)
):
    trial = db.get(
        ClinicalTrialDB,
        trial_id
    )

    if trial is None:
        raise HTTPException(
            status_code=404,
            detail="Clinical trial not found"
        )

    return trial
```





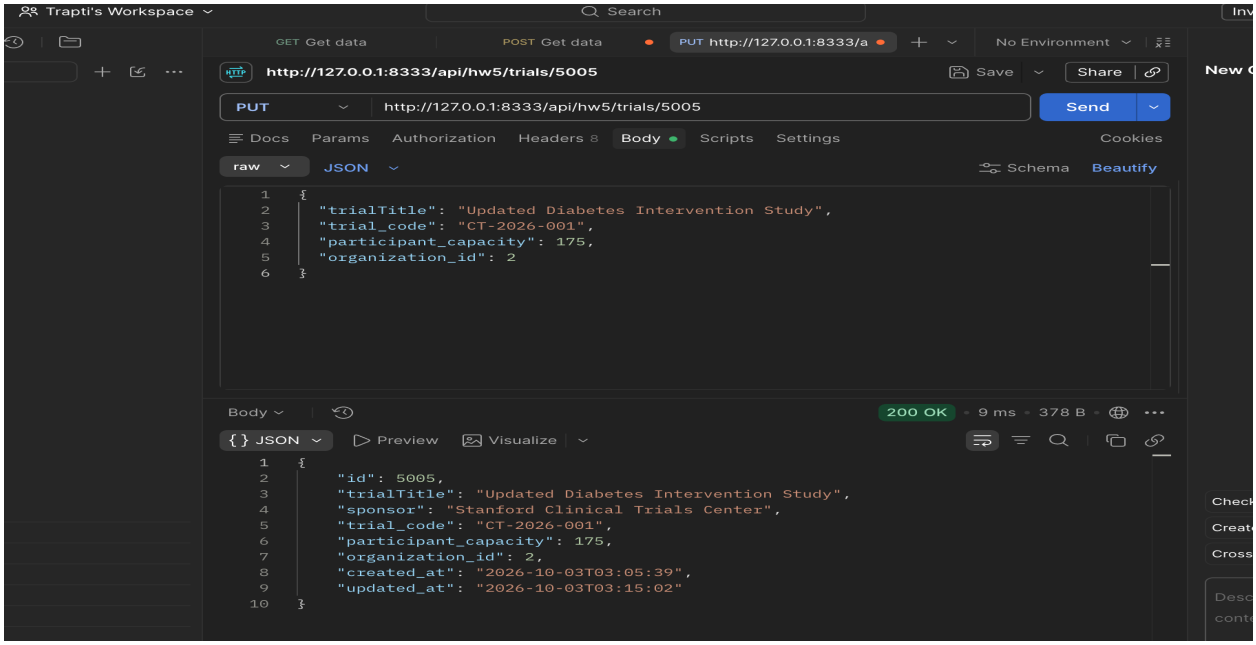
1.4.4 Update Clinical Trial

```python
    def update_trial(
        trial_id: int,
        trial_data: HW5TrialUpdate,
        db: Session = Depends(get_db)
    ):
        trial = db.get(
            ClinicalTrialDB,
            trial_id
        )

        if trial is None:
            raise HTTPException(
                status_code=404,
                detail="Clinical trial not found"
            )
```

I implemented the PUT endpoint to update an existing clinical trial. The endpoint

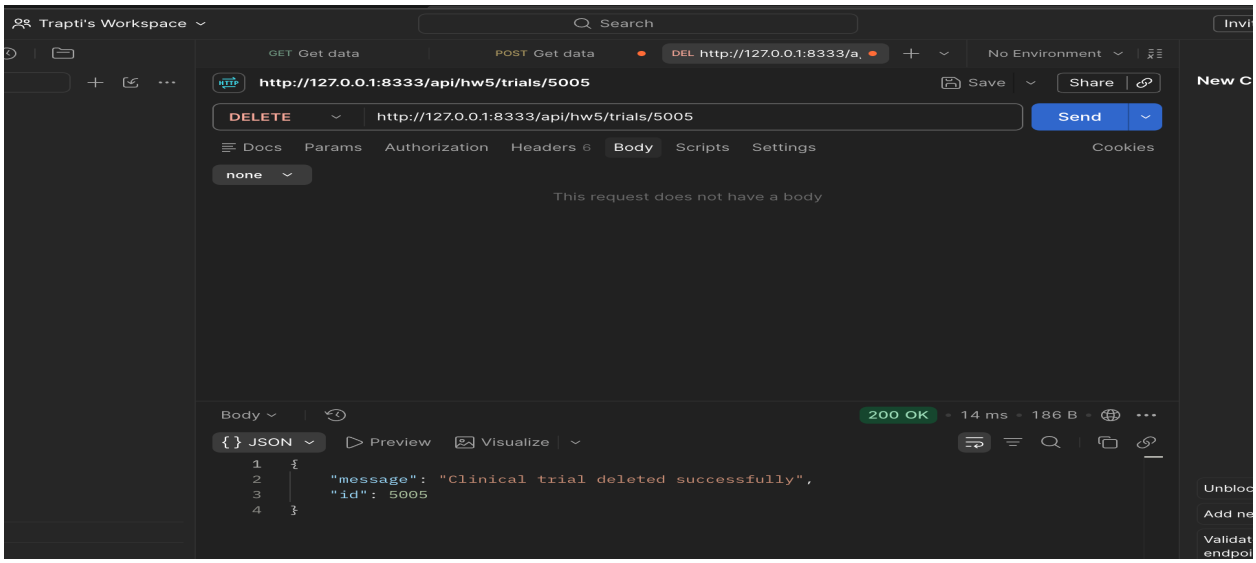
validates that both the clinical trial and the selected research organization exist before saving the changes. The updated trial title, trial code, participant capacity, and organization relationship are then persisted in MySQL.



1.4.5 Delete Clinical Trial:

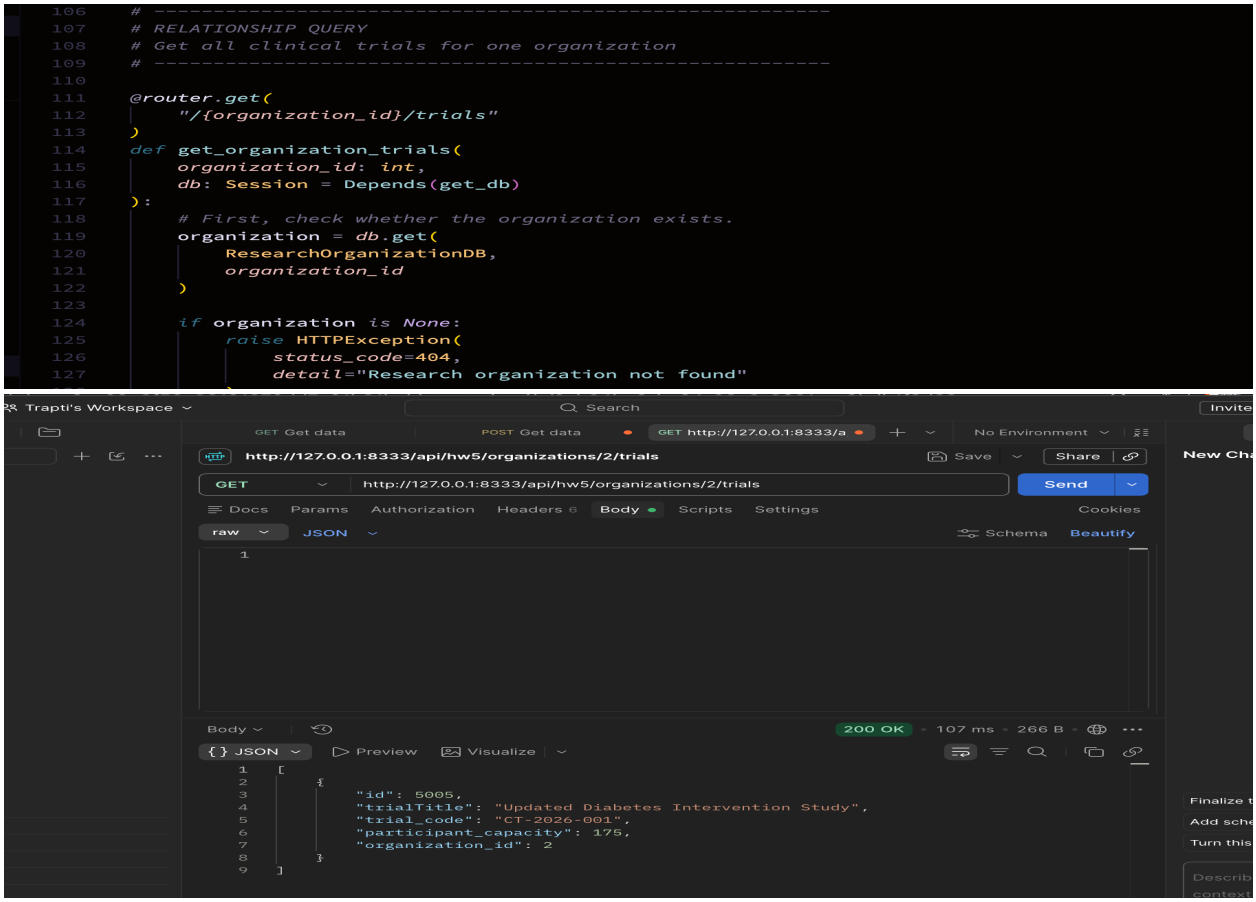
I implemented the DELETE endpoint to remove an existing clinical trial using its ID. The endpoint first verifies that the trial exists, deletes it from MySQL, and returns a successful confirmation response.

```
256    def delete_trial(
257        trial_id: int,
258        db: Session = Depends(get_db)
259    ):
260        trial = db.get(
261            ClinicalTrialDB,
262            trial_id
263        )
264
265        if trial is None:
266            raise HTTPException(
267                status_code=404,
268                detail="Clinical trial not found"
269            )
270
271        db.delete(trial)
272        db.commit()
```

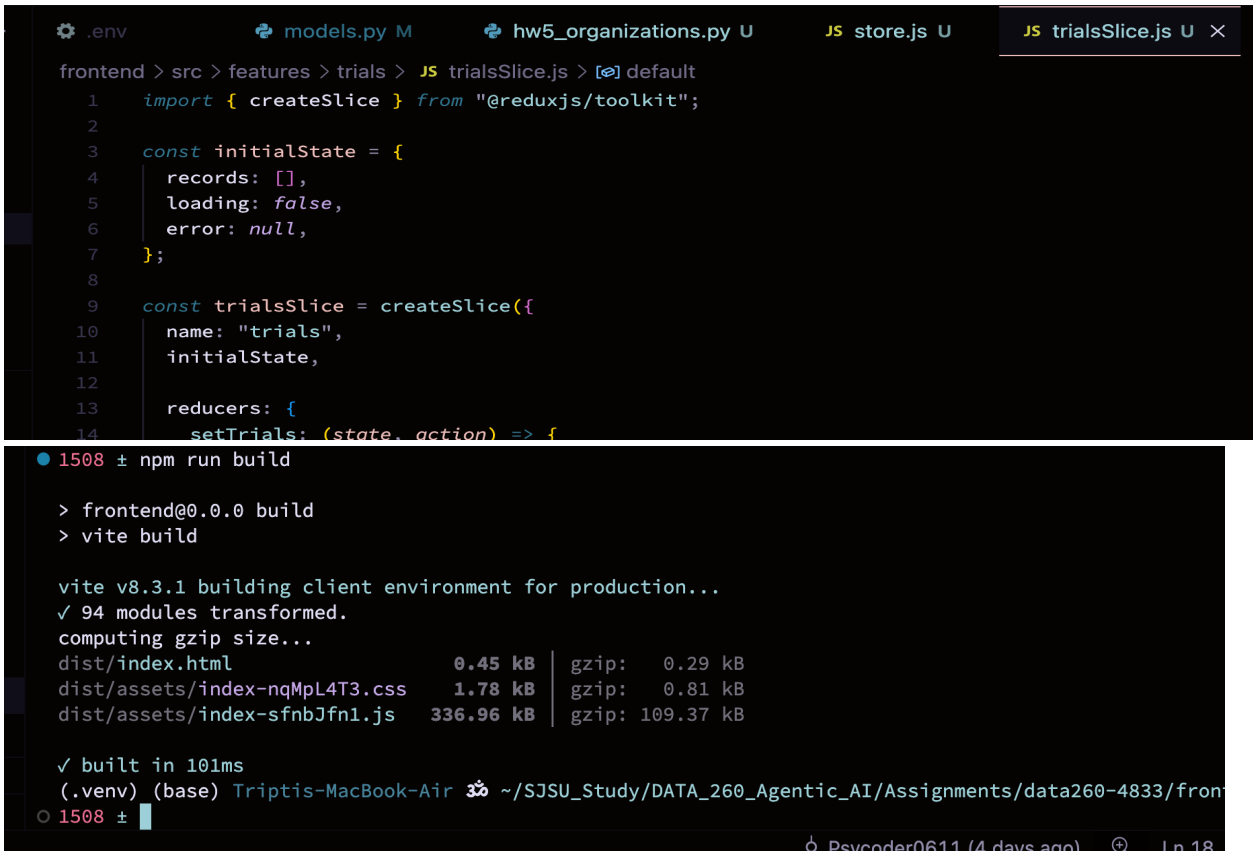


1.5 Relationship Query:

I implemented a relationship endpoint to retrieve all clinical trials associated with a specific research organization. The endpoint first verifies that the organization exists and then returns the trials connected through the organization_id foreign-key relationship.



1.6 Redux Store and Slice:



```
.env  models.py M  hw5_organizations.py U  store.js U  trialsSlice.js U
frontend > src > features > trials > trialsSlice.js > default
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  records: [],
  loading: false,
  error: null,
};

const trialsSlice = createSlice({
  name: "trials",
  initialState,

  reducers: {
    setTrials: (state, action) => {
```

```
1508 ± npm run build

> frontend@0.0.0 build
> vite build

vite v8.3.1 building client environment for production...
✓ 94 modules transformed.
computing gzip size...
dist/index.html                   0.45 kB | gzip:   0.29 kB
dist/assets/index-nqMpL4T3.css    1.78 kB | gzip:   0.81 kB
dist/assets/index-sfnbJfn1.js   336.96 kB | gzip: 109.37 kB

✓ built in 101ms
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833/fron
1508 ±
```

I configured Redux Toolkit as the frontend state-management layer and created a trials slice for the Clinical Trial entity. The Redux state stores the clinical-trial records together with loading and error status. Reducers were defined to set, add, update, and remove trial records. I also connected the Redux store to the React application using the Provider component so that Redux state can be accessed by all frontend pages.

1.7 Async Thunks:



```
// ------------------------------------------------------------
// REDUX SLICE
// ------------------------------------------------------------

const trialsSlice = createSlice({
  name: "trials",
  initialState,

  reducers: {},

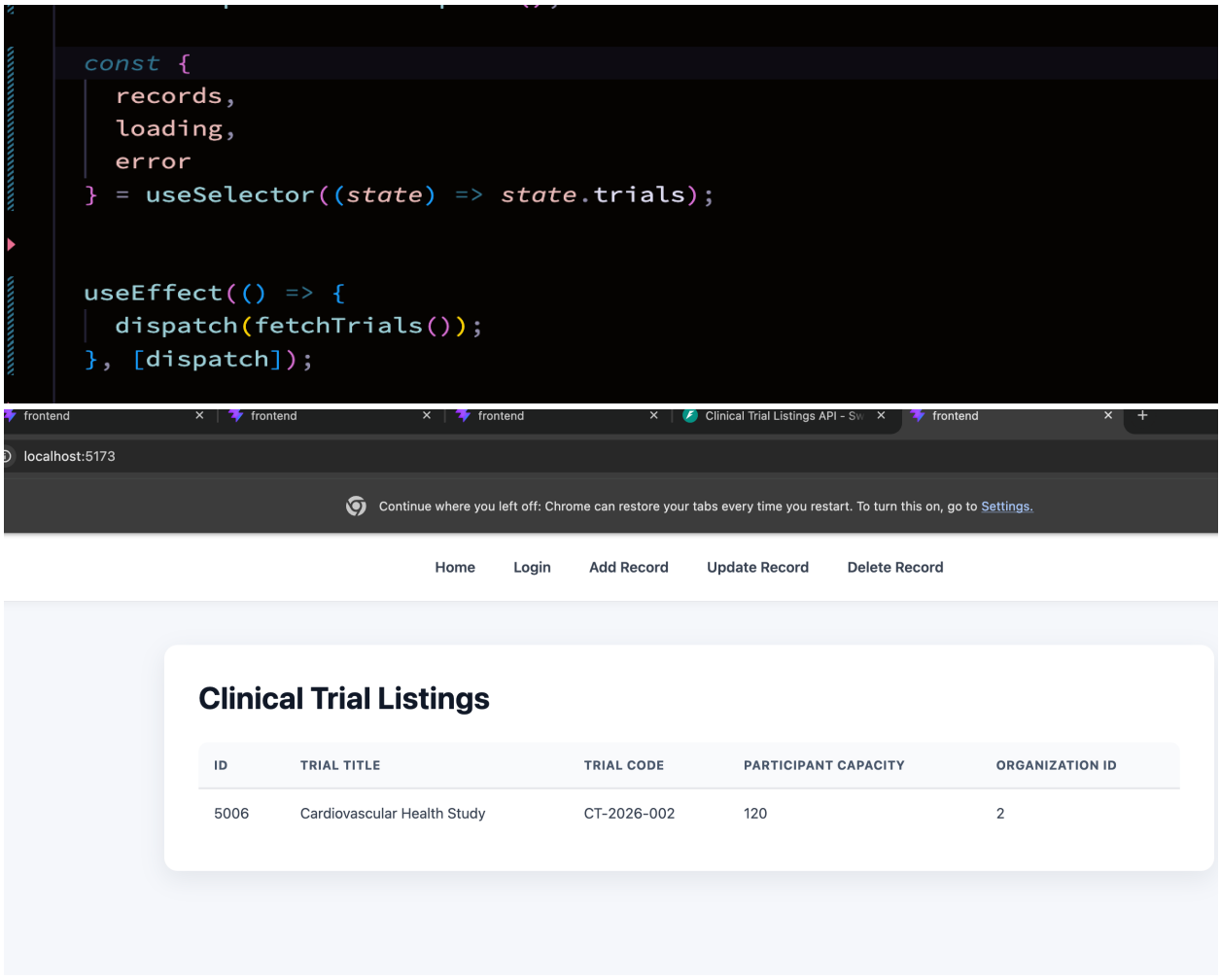
  extraReducers: (builder) => {
    builder

      // FETCH
      .addCase(fetchTrials.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchTrials.fulfilled, (state, action) => {
        state.loading = false;
        state.records = action.payload;
      })
      .addCase(fetchTrials.rejected, (state, action) => {
        state.loading = false;
        state.error = action.payload;
      })
```



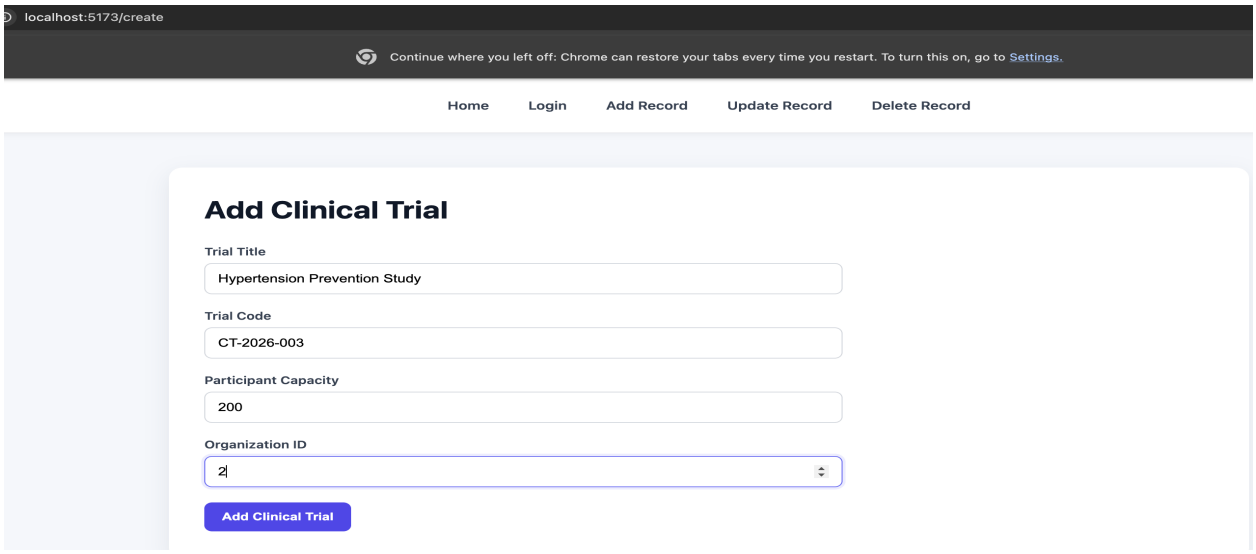
I implemented Redux Toolkit async thunks using Axios for all four Clinical Trial CRUD operations. fetchTrials retrieves the trial list, createTrial sends new trial data, editTrial updates an existing record by ID, and deleteTrial removes a record by ID. Each thunk also uses rejectWithValue so API errors can be stored and displayed through Redux state.

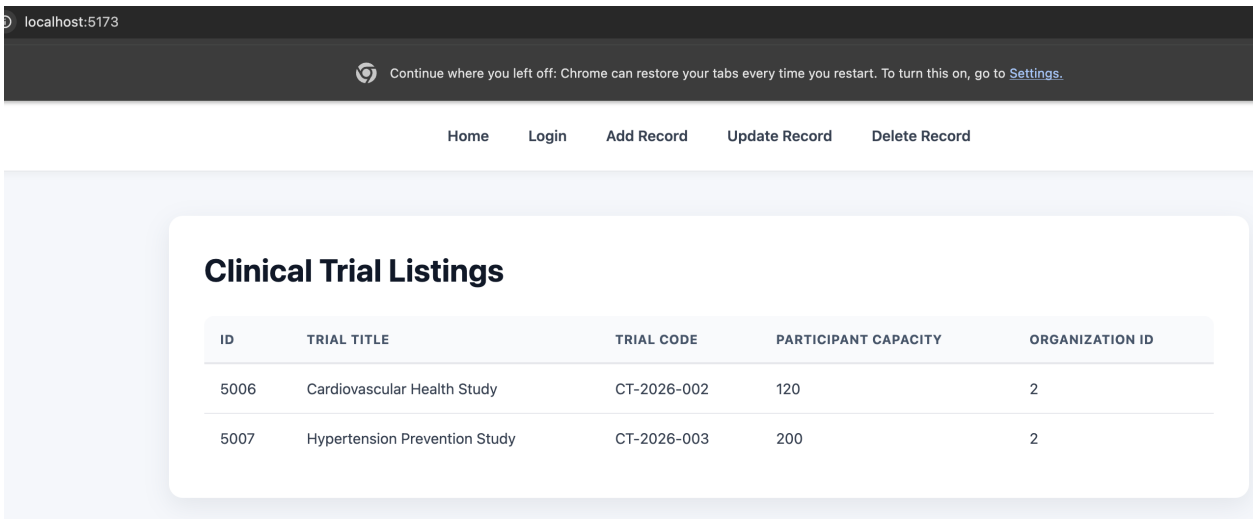
1.8 Home Screen:



I updated the Home screen to use Redux state for displaying Clinical Trial data. The component dispatches the fetchTrials async thunk when it loads, and the returned records are stored in Redux. The UI then reads those records using useSelector and displays the trial ID, title, trial code, participant capacity, and organization ID in a table.

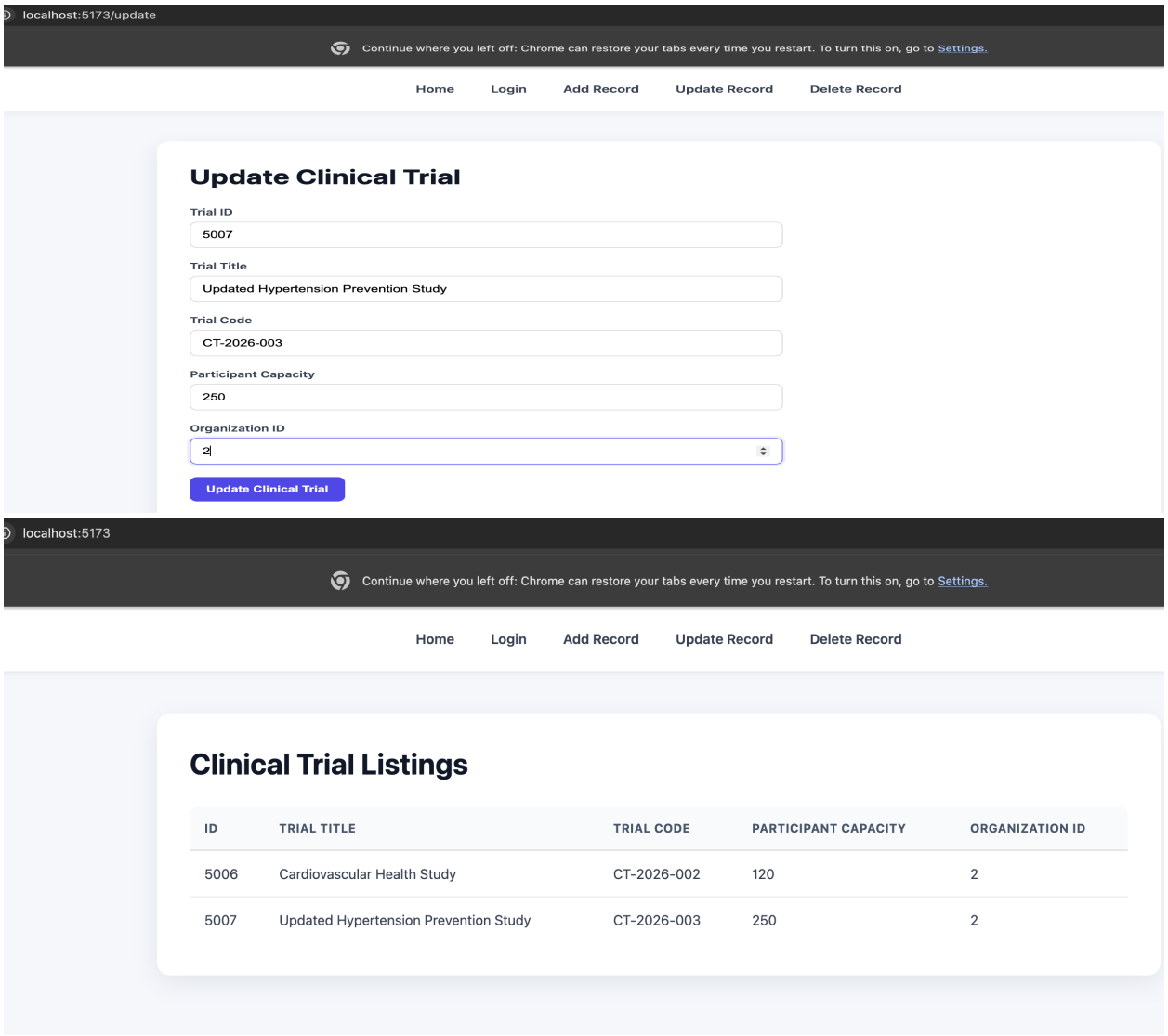
1.9 Create:





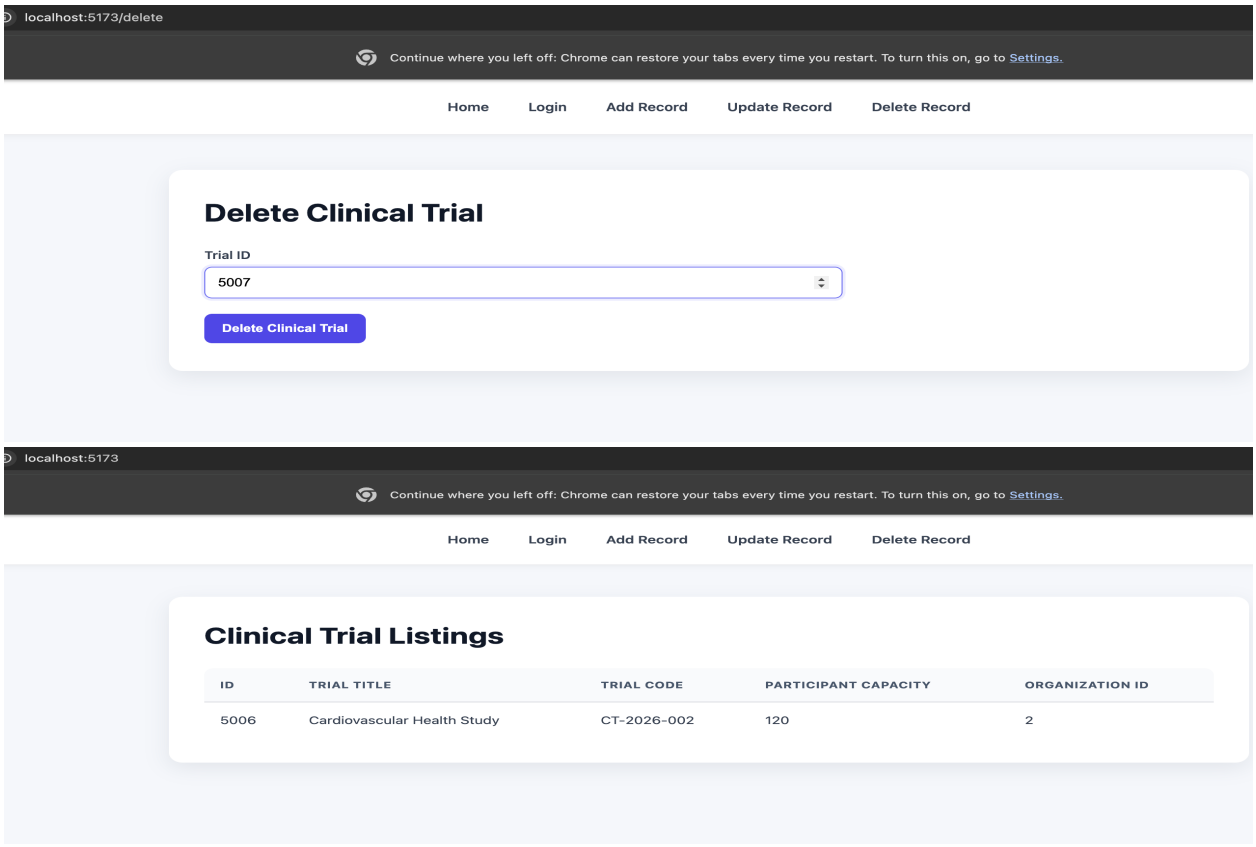
I updated the Create page to use the Redux createTrial async thunk instead of making a direct Axios request from the component. The form collects the trial title, unique trial code, participant capacity, and organization ID. After a successful Redux dispatch, the new clinical trial is added to the application state and the user is redirected to the Home screen.

1.10 Update:



I updated the Update page to dispatch the Redux editTrial async thunk. The form collects the trial ID and revised trial information, including title, trial code, participant capacity, and organization ID. After a successful PUT request, Redux updates the corresponding record in state and the user is redirected to the Home screen.

1.11 Delete:



I updated the Delete page to use the Redux deleteTrial async thunk. The user enters the clinical trial ID, and the component dispatches the delete action to the Redux data layer. After the backend confirms deletion, Redux removes the corresponding record from state and the user is redirected to the Home screen.

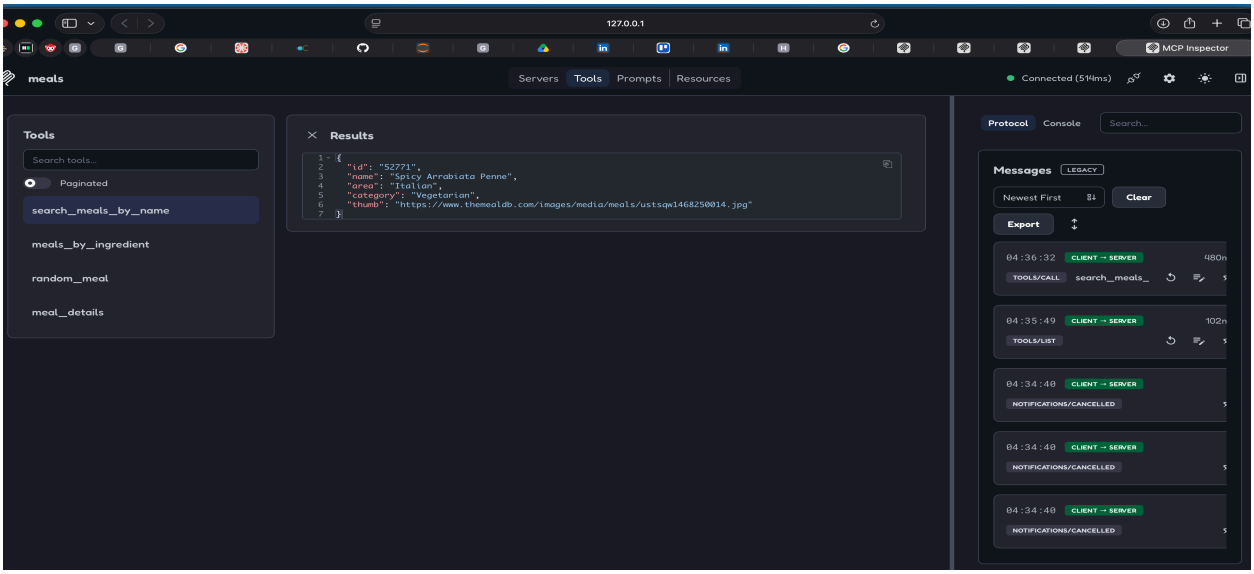
Part 2 – MCP Tool Servers

2.1 TheMealDB MCP Server:

I created a local MCP server named meals using the MCP 2.x MCPServer interface (the current equivalent of FastMCP). The server exposes four tools for searching meals by name, filtering meals by ingredient, retrieving a random meal, and retrieving full recipe details by meal ID. The server runs over STDIO, and logging is directed to stderr to avoid corrupting the MCP JSON-RPC stream.

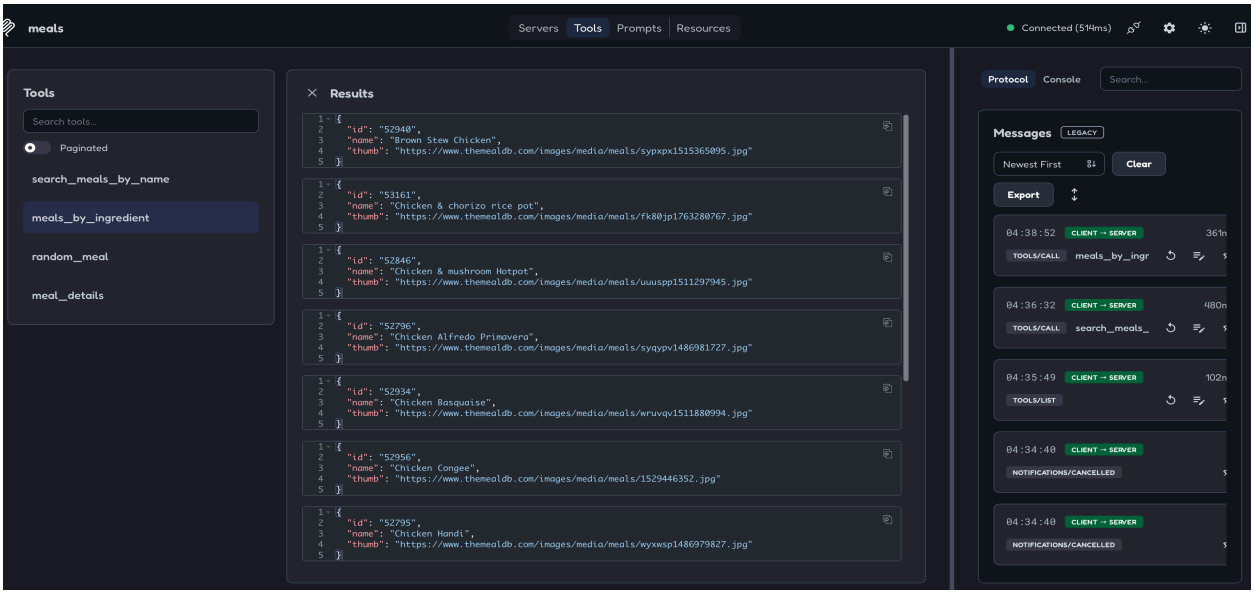
2.1.1 Search Meals by Name:

I tested the search_meals_by_name tool in MCP Inspector using the query Arrabiata. The tool successfully called TheMealDB and returned the meal ID, name, cuisine area, category, and thumbnail URL.

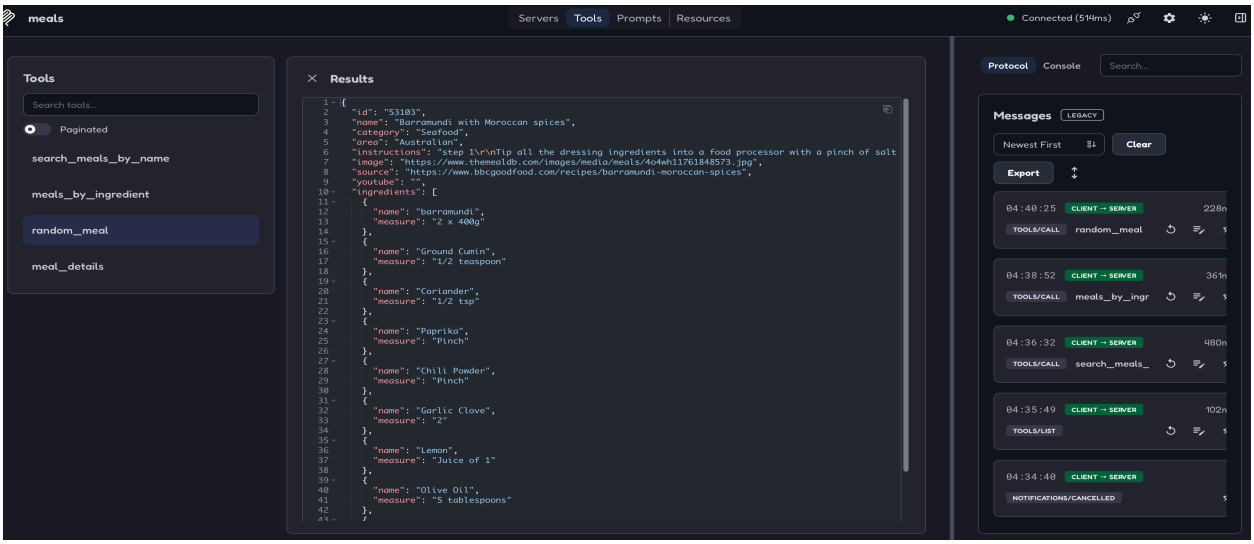


2.1.2 Meals by Ingredient:

I tested the meals_by_ingredient MCP tool using chicken as the main ingredient with a limit of 12 results. The tool successfully called TheMealDB filter endpoint and returned matching meals with the meal ID, name, and thumbnail URL, which matches the required output shape.



2.1.3 Random Meal:



I tested the random_meal MCP tool in MCP Inspector. The tool successfully retrieved one random recipe from TheMealDB and returned the full meal details, including category, cuisine area, instructions, image, source links, and ingredient-measure pairs.

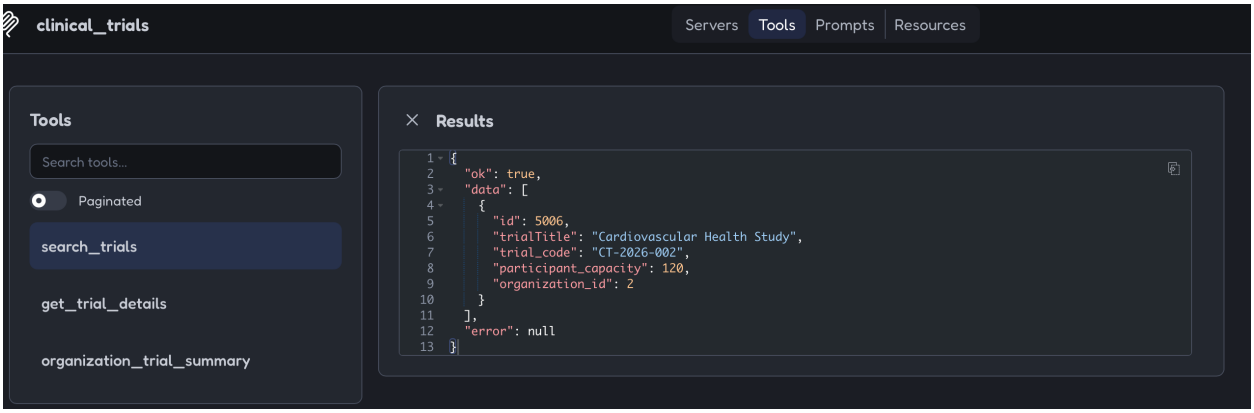
2.1.4 Meal Details:

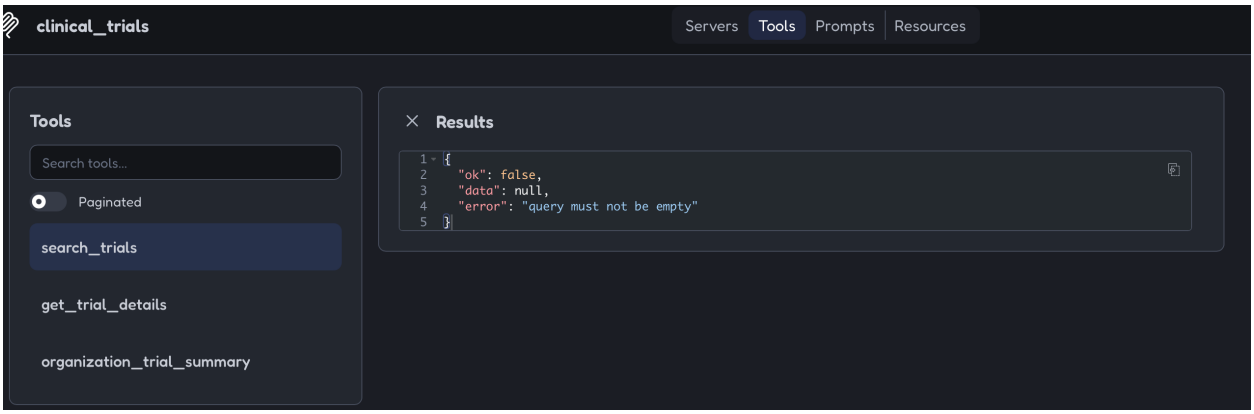
I tested the meal_details MCP tool using meal ID 52771, which corresponds to Spicy Arrabiata Penne. The tool successfully returned the full recipe details, including category, cuisine area, cooking instructions, image and source information, and ingredient-measure pairs.



2.2 Domain MCP Server:

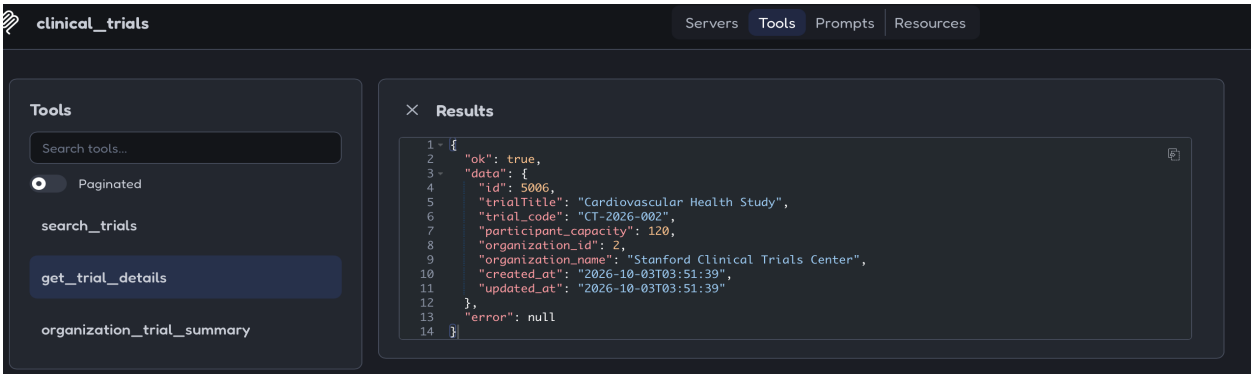
2.2.1 search_trials:

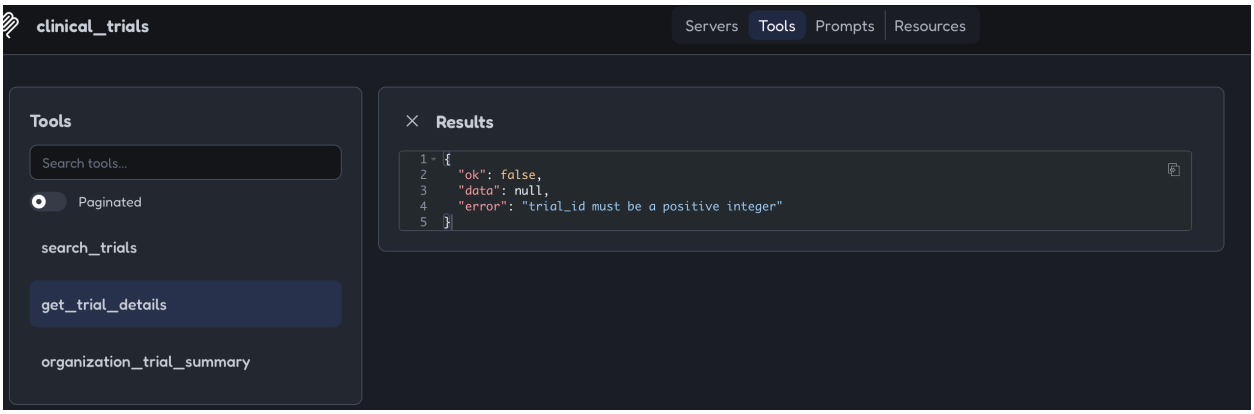




I tested the search_trials domain MCP tool with a valid title query and confirmed that it returned matching clinical trials inside the common {ok, data, error} response envelope. I also tested an intentionally invalid empty query, which returned ok: false with a clear validation error instead of crashing.

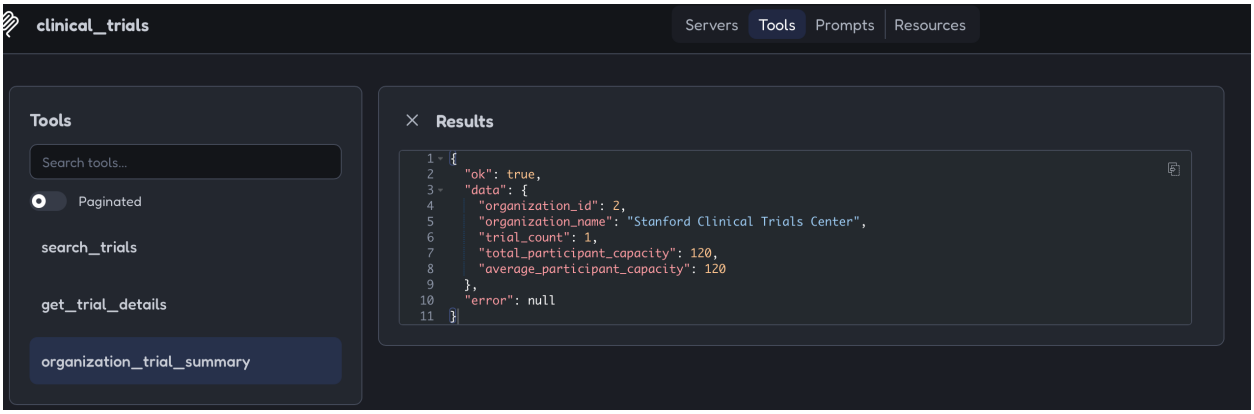
2.2.2 get_trial_details:

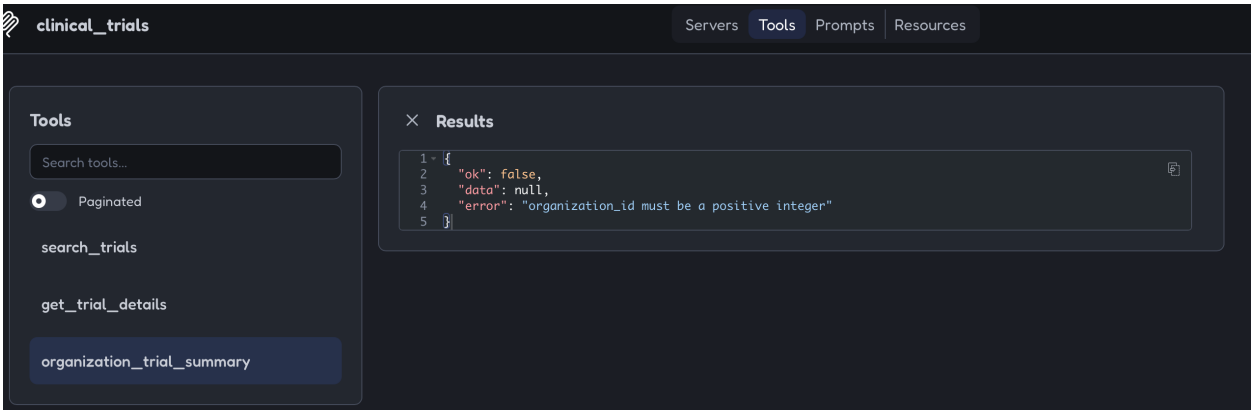




I tested the get_trial_details MCP tool with a valid clinical trial ID and confirmed that it returned the complete trial record inside the common {ok, data, error} response envelope. I also tested the invalid value trial_id = 0, which was rejected with a clear validation error because the trial ID must be a positive integer.

2.2.3 organization_trial_summary:





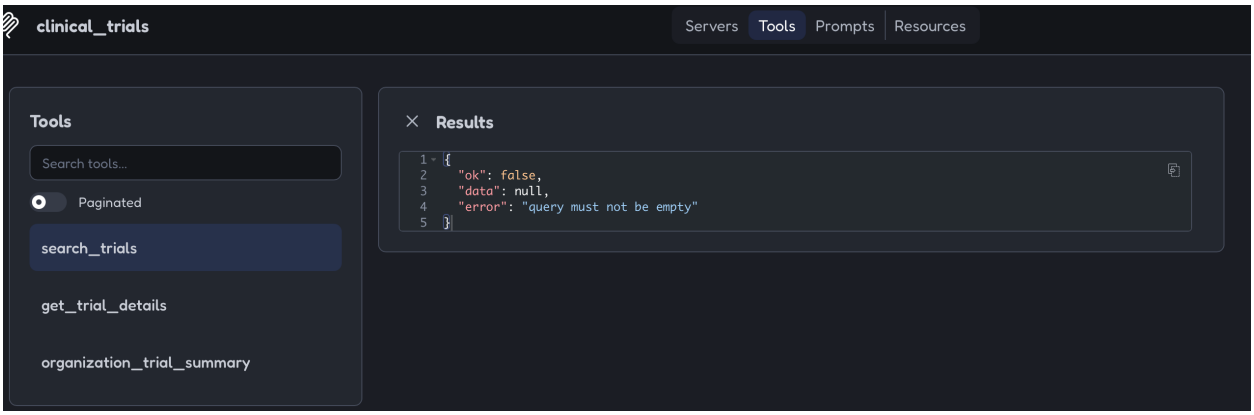
I tested the organization_trial_summary MCP tool with a valid research organization ID and confirmed that it returned aggregate information including the number of associated trials, total participant capacity, and average participant capacity. I also

tested the invalid value organization_id = 0, which was rejected with a clear validation error inside the common {ok, data, error} response envelope.

Part 3 – Tool Contracts Under Stress

3.1 Tool Contracts and Rejected Inputs

3.1.1 search_trials:



Expected JSON input:

```
{
  "query": "string",
  "limit": "integer between 1 and 25"
}
```

Rejected input:

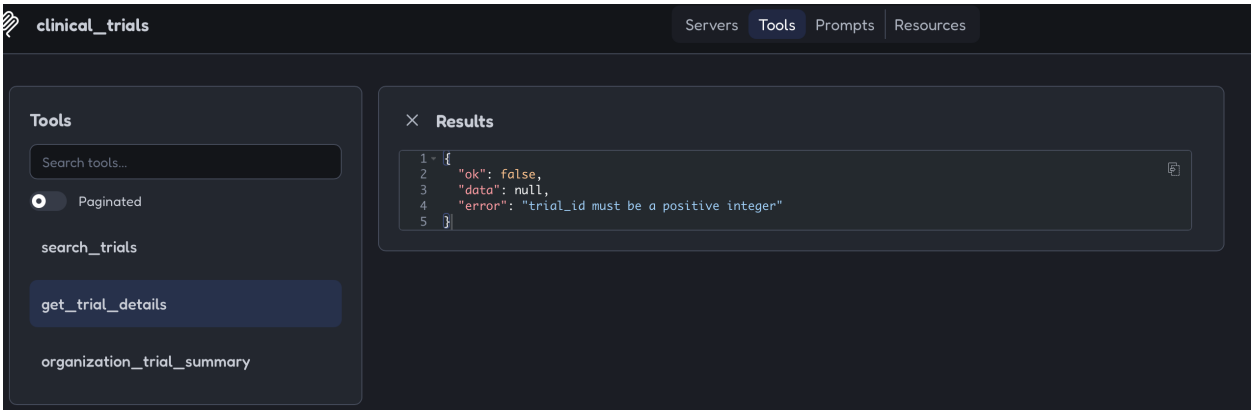
```
{
  "query": " ",
  "limit": 10
}
```

Returned error:

```
{
  "ok": false,
  "data": null,
  "error": "query must not be empty"
}
```

The input was rejected because the search query contained only whitespace. The tool strips whitespace before validation, leaving an empty query, which is not allowed.

3.1.2 get_trial_details:



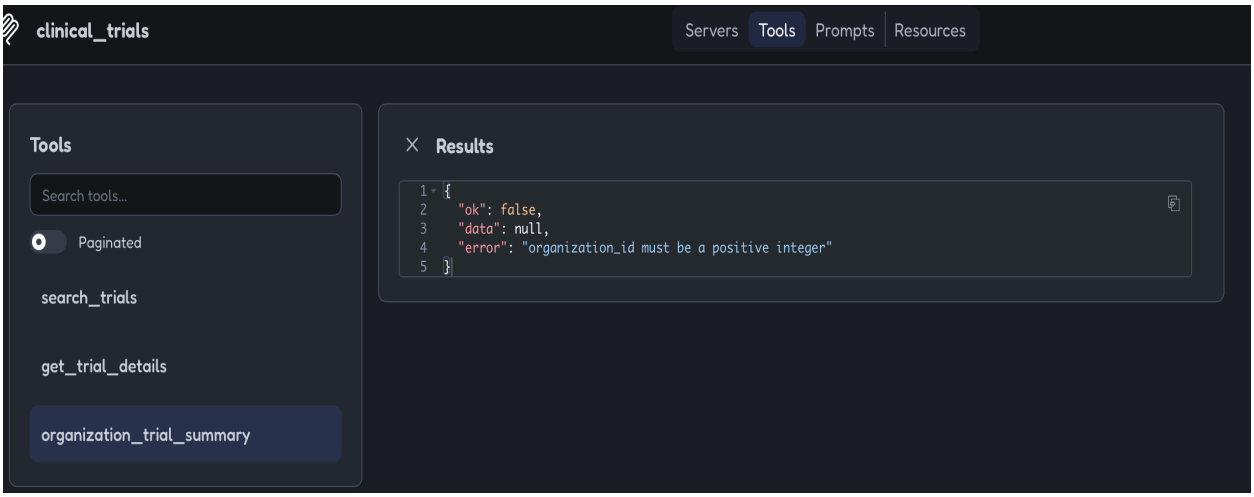
The input was rejected because a clinical trial ID must be a positive integer. The value 0 is outside the valid range.

```
Expected JSON input:
{
  "trial_id": "positive integer"
}

Rejected input:
{
  "trial_id": 0
}

Returned error:
{
  "ok": false,
  "data": null,
  "error": "trial_id must be a positive integer"
}
```

3.1.3 organization_trial_summary:



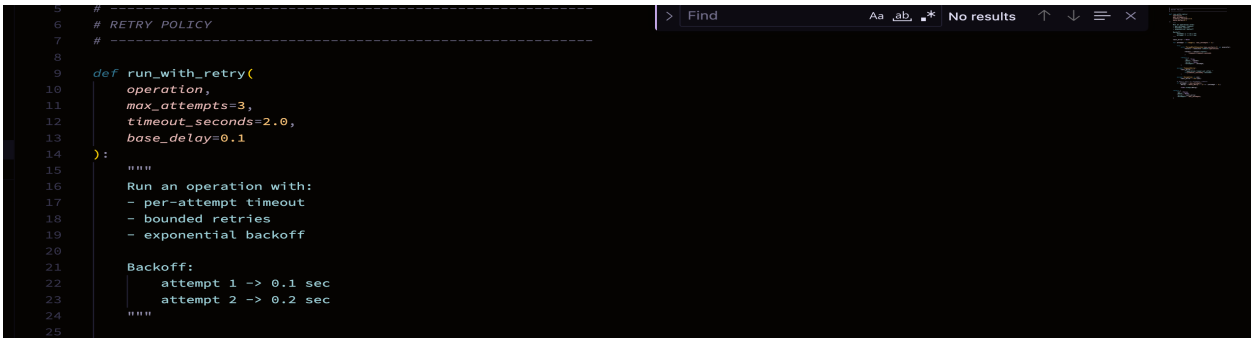
The input was rejected because the organization ID must be a positive integer. The value 0 fails this validation rule.

```
Expected JSON input:
{
  "organization_id": "positive integer"
}

Rejected input:
{
  "organization_id": 0
}

Returned error:
{
  "ok": false,
  "data": null,
  "error": "organization_id must be a positive integer"
}
```

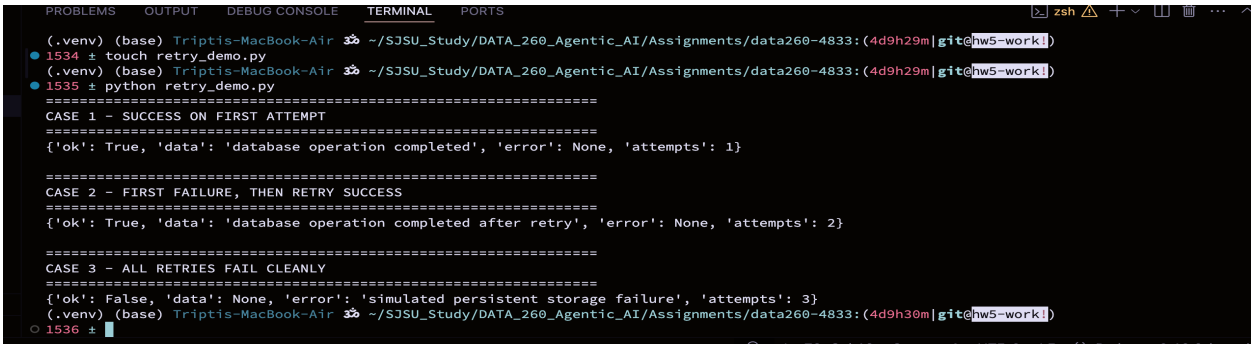
3.2 Timeout and Bounded Exponential-Backoff Retry Policy:



```python
# ------------------------------------------------------------
# RETRY POLICY
# ------------------------------------------------------------

def run_with_retry(
    operation,
    max_attempts=3,
    timeout_seconds=2.0,
    base_delay=0.1
):
    """
    Run an operation with:
    - per-attempt timeout
    - bounded retries
    - exponential backoff

    Backoff:
        attempt 1 -> 0.1 sec
        attempt 2 -> 0.2 sec
    """
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

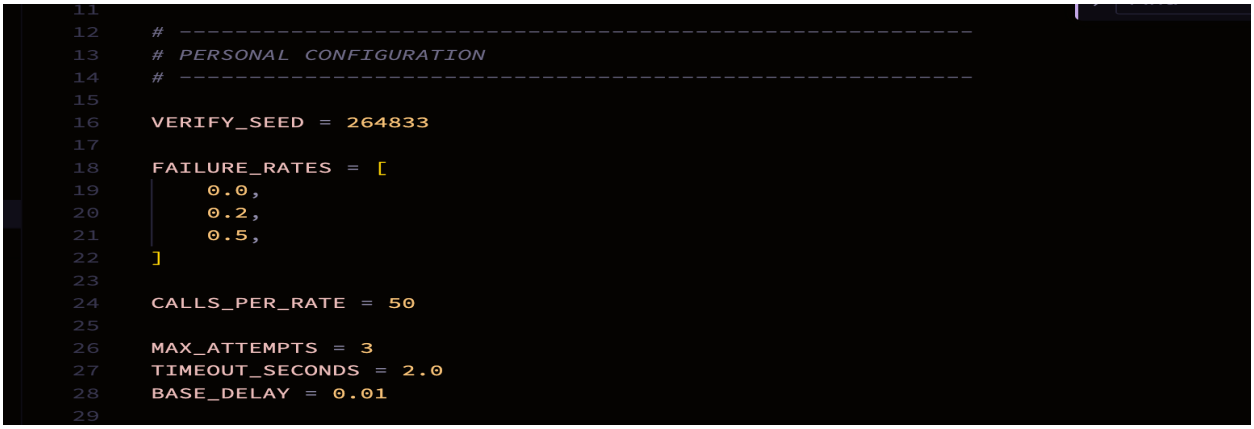
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(4d9h29m|git@hw5-work)
1534 ± touch retry_demo.py
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(4d9h29m|git@hw5-work)
1535 ± python retry_demo.py
============================================================
CASE 1 - SUCCESS ON FIRST ATTEMPT
============================================================
{'ok': True, 'data': 'database operation completed', 'error': None, 'attempts': 1}

============================================================
CASE 2 - FIRST FAILURE, THEN RETRY SUCCESS
============================================================
{'ok': True, 'data': 'database operation completed after retry', 'error': None, 'attempts': 2}

============================================================
CASE 3 - ALL RETRIES FAIL CLEANLY
============================================================
{'ok': False, 'data': None, 'error': 'simulated persistent storage failure', 'attempts': 3}
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(4d9h30m|git@hw5-work)
1536 ±
```

I implemented a bounded retry policy with a per-attempt timeout and exponential backoff. Each operation can make at most three attempts. When an attempt fails, the delay increases exponentially before the next retry. I demonstrated three behaviors: an operation succeeding on the first attempt, a temporary failure succeeding on the second attempt, and a persistent failure exhausting all retries and returning a clean error result instead of crashing.

3.3 Deterministic Failure Simulation:



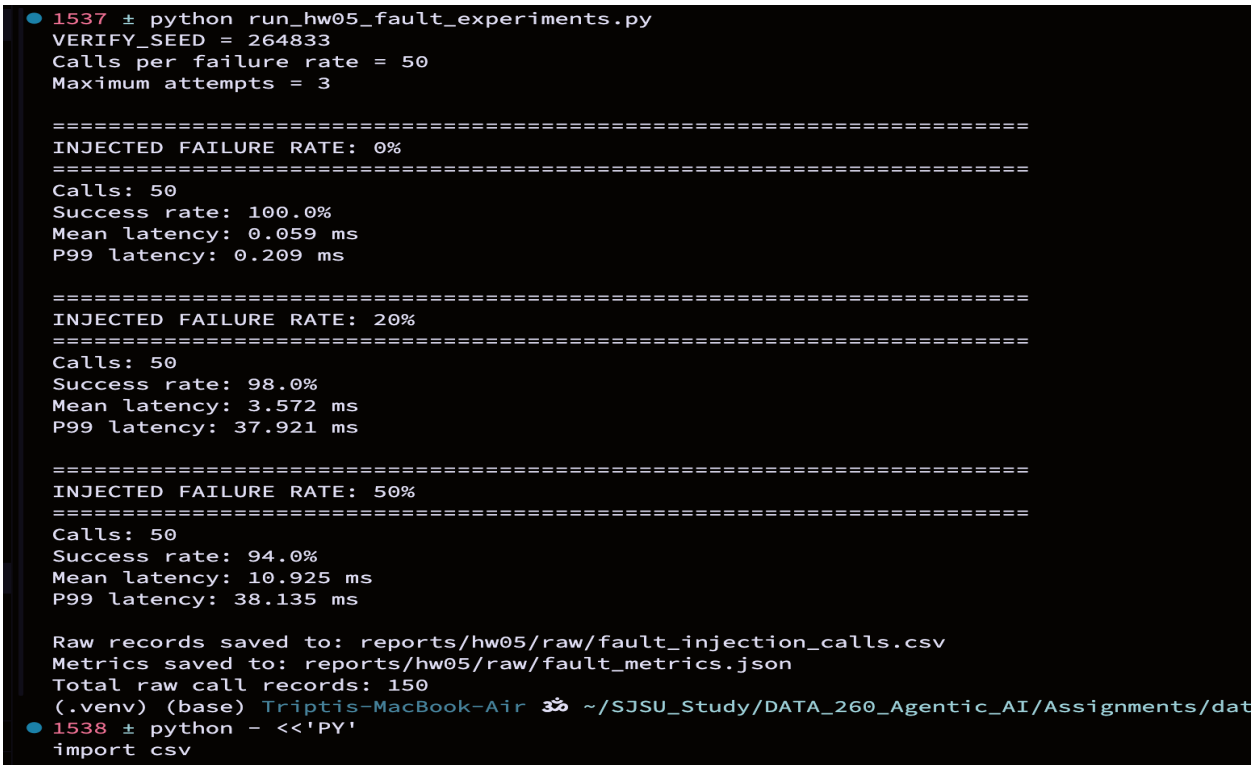
```python
# ------------------------------------------------------------
# PERSONAL CONFIGURATION
# ------------------------------------------------------------

VERIFY_SEED = 264833

FAILURE_RATES = [
    0.0,
    0.2,
    0.5,
]

CALLS_PER_RATE = 50

MAX_ATTEMPTS = 3
TIMEOUT_SECONDS = 2.0
BASE_DELAY = 0.01
```



Injected Failure Rate	Success Rate	Mean Latency (ms)	p99 Latency (ms)
0%	100.0%	0.058	0.200
20%	98.0%	3.512	37.778
50%	94.0%	11.039	38.326

The retry policy maintained a high final success rate even as the injected failure rate increased. At 0% injected failure, all calls succeeded immediately with very low latency. At 20% and 50% injected failure, retries recovered most transient failures, but average and p99 latency increased because some calls required additional attempts and backoff delays.

3.4 Retry Policy Evaluation:

The retry policy is suitable for an interactive assistant because it maintains a high success rate while keeping retry delays bounded. At 0% injected failure, all calls succeeded with negligible latency. At 20% failure, the final success rate remained 98.0%, while mean latency increased to 3.512 ms because some calls required retries. At 50% failure, the policy still achieved a 94.0% success rate, but mean latency increased further to 11.039 ms. The p99 latency was approximately 37.778 ms at 20% failure and 38.326 ms at 50% failure, showing the additional latency introduced by retries and exponential backoff.

For an interactive assistant, I would keep the retry count small and the timeout relatively short so the user does not wait too long for a response. For batch processing, where longer execution time is acceptable, I would allow more retry attempts, use a longer timeout, and increase the backoff delays. This would improve the chance of recovery from temporary failures at the cost of higher total processing time.

Part 4 – Safe Tool Entry Point and Offline Tests

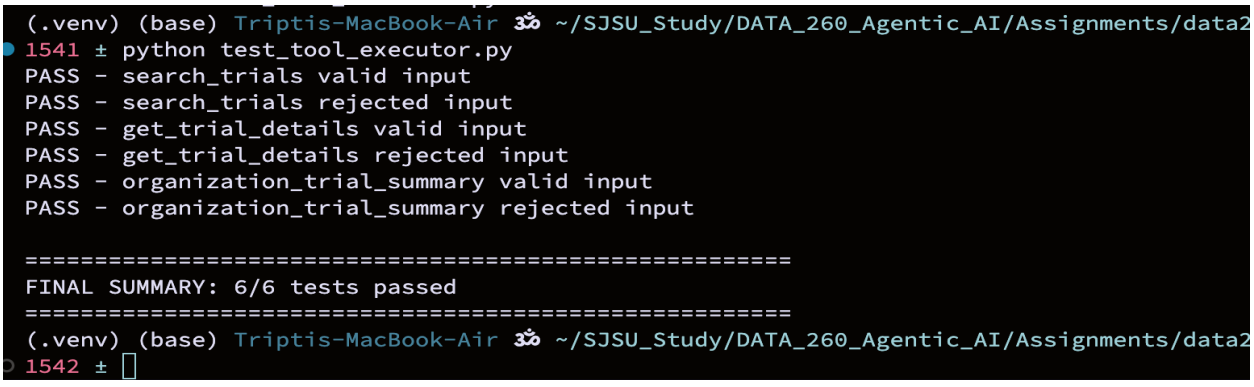
4.1 Single Tool Entry Point:



```
246     # SINGLE SAFE TOOL ENTRY POINT
247     # ------------------------------------------------------------
248
249     def execute_tool(
250         name: str,
251         inputs: Dict[str, Any],
252         tool_registry: Dict[
253             str,
254             Callable[[dict], dict]
255         ] | None = None,
256     ):
257         """
258         Single entry point for all domain tools.
259
260         Returns a JSON string and never crashes
261         on ordinary tool errors.
262         """
263
264         registry = (
265             tool_registry
266             if tool_registry is not None
267             else DEFAULT_TOOL_REGISTRY
268         )
269
270         try:
271             if name not in registry:
272                 result = error_response(
```

I implemented execute_tool(name, inputs) as the single entry point for the three domain tools. It selects the requested tool from a registry, preserves the common {ok, data, error} envelope, serializes the result as a JSON string, and catches unexpected exceptions so tool failures do not crash the caller.

4.2 Offline Tests:



```
(.venv) (base) Triptis-MacBook-Air ॐ ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data2
1541 ± python test_tool_executor.py
PASS - search_trials valid input
PASS - search_trials rejected input
PASS - get_trial_details valid input
PASS - get_trial_details rejected input
PASS - organization_trial_summary valid input
PASS - organization_trial_summary rejected input

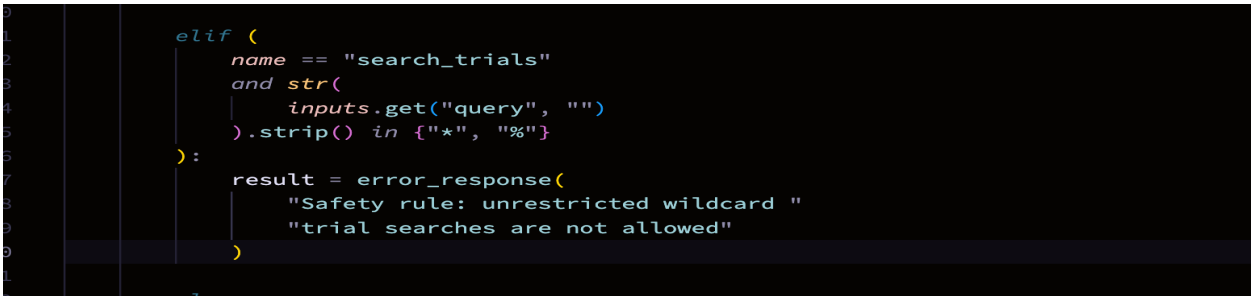
============================================================
FINAL SUMMARY: 6/6 tests passed
============================================================
(.venv) (base) Triptis-MacBook-Air ॐ ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data2
1542 ±
```

I created an offline assert-based test runner with six tests: one valid and one rejected input for each of the three domain tools. The rejected cases reuse the same invalid examples documented in Part 3. Dependency injection through a mock tool registry allows the test suite to run repeatedly without a live database, external API, or LLM.

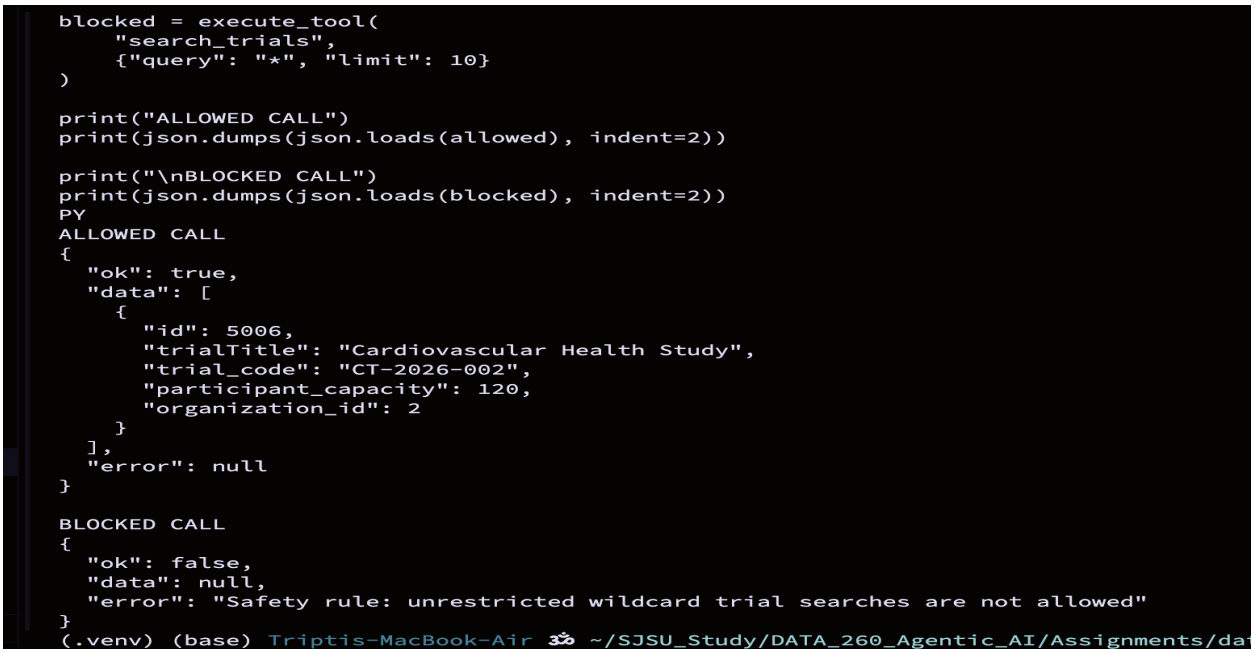
Part 5 – Agent Loop, Safety Rule, and Reflection

5.1 Safety Rule:

I added a domain-specific safety rule inside execute_tool. The rule blocks unrestricted wildcard trial searches using * or %, which could otherwise be used to enumerate the entire clinical-trial dataset. Targeted searches such as Cardio remain allowed. When the rule is violated, the executor returns the standard {ok: false, data: null, error: "..."} envelope without raising an exception.



```
        elif (
            name == "search_trials"
            and str(
                inputs.get("query", "")
            ).strip() in {"*", "%"}
        ):
            result = error_response(
                "Safety rule: unrestricted wildcard "
                "trial searches are not allowed"
            )
```



```
blocked = execute_tool(
    "search_trials",
    {"query": "*", "limit": 10}
)

print("ALLOWED CALL")
print(json.dumps(json.loads(allowed), indent=2))

print("\nBLOCKED CALL")
print(json.dumps(json.loads(blocked), indent=2))
PY
ALLOWED CALL
{
  "ok": true,
  "data": [
    {
      "id": 5006,
      "trialTitle": "Cardiovascular Health Study",
      "trial_code": "CT-2026-002",
      "participant_capacity": 120,
      "organization_id": 2
    }
  ],
  "error": null
}

BLOCKED CALL
{
  "ok": false,
  "data": null,
  "error": "Safety rule: unrestricted wildcard trial searches are not allowed"
}
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/da
```

5.2 Agent Loop:

I implemented run_agent(user_input) using the local Ollama model qwen3:8b and the three domain tools: search_trials, get_trial_details, and organization_trial_summary. The agent processes the user request step by step, asks the model whether to call a tool or return a final answer, and routes all tool calls through the single execute_tool() entry point. A max_steps limit prevents the agent from running indefinitely. Each step records the tool name, tool inputs, tool result, final answer, and stop reason in reports/hw05/raw/agent_runs.jsonl. The possible stop reasons include normal completion, safety-rule block, model error, and reaching the max_steps limit.

```
127    # ------------------------------------------------------------
128    # AGENT LOOP
129    # ------------------------------------------------------------
130
131    def run_agent(
132        user_input,
133        model=None,
134        max_steps=MAX_STEPS,
135        tool_executor=execute_tool,
136    ):
137        if model is None:
138            model = OllamaModel()
139
140        messages = [
141            {
142                "role": "system",
143                "content": SYSTEM_PROMPT,
144            },
145            {
146                "role": "user",
147                "content": user_input,
148            },
149        ]
150
151        tool_call_count = 0
```

5.3 Additional Offline Tests:

I extended the offline test suite with two additional tests. The first verifies that execute_tool blocks an unrestricted wildcard search through the domain safety rule. The second uses a MockModel that continuously requests tool calls and confirms that run_agent stops after reaching max_steps. A temporary JSONL log path and mock tool executor keep this test completely offline without a database, external API, or live Ollama model.

```python
def test_safety_rule_block():
    result = json.loads(
        execute_tool(
            "search_trials",
            {
                "query": "*",
                "limit": 10,
            },
            MOCK_REGISTRY,
        )
    )

    assert result["ok"] is False
    assert result["data"] is None
    assert (
        result["error"]
        ==
        "Safety rule: unrestricted wildcard "
        "trial searches are not allowed"
    )
```

```python
# ------------------------------------------------------------
# MOCK MODEL FOR MAX_STEPS TEST
# ------------------------------------------------------------

class MockModel:
    """
    Always asks for another valid tool call.
    This forces the agent loop to reach max_steps.
    """

    def generate(self, messages):
        return json.dumps(
            {
                "action": "tool",
                "tool": "search_trials",
                "inputs": {
                    "query": "Cardio",
                    "limit": 10,
                },
            }
        )


def mock_agent_tool_executor(
    name,
    inputs
):
```

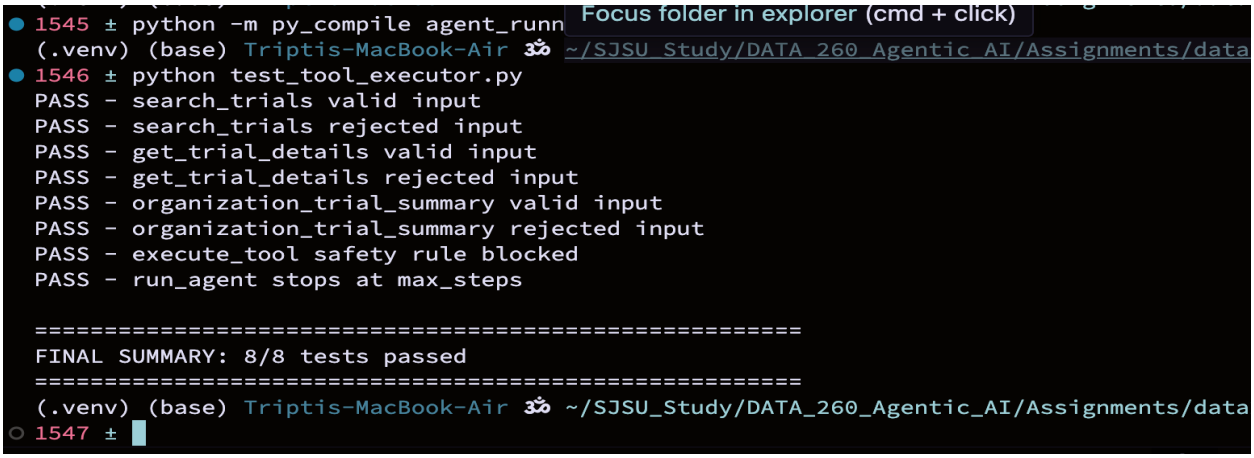
```python
def test_agent_max_steps():
    original_log_path = (
        agent_runner.LOG_PATH
    )

    try:
        with tempfile.TemporaryDirectory() as temp_dir:

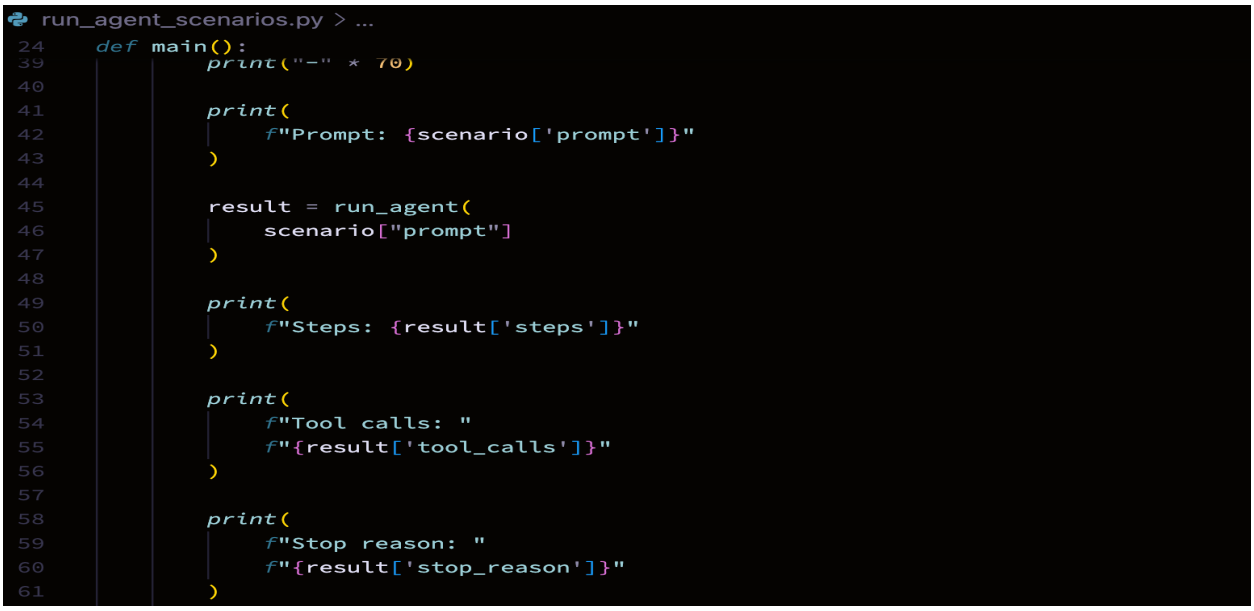
            agent_runner.LOG_PATH = (
                Path(temp_dir)
                / "agent_runs.jsonl"
            )

            result = run_agent(
                "Keep searching for trials",
                model=MockModel(),
                max_steps=3,
                tool_executor=
                    mock_agent_tool_executor,
            )

            assert (
                result["stop_reason"]
                == "max_steps"
            )
```



5.4 Local Ollama Scenarios and Metrics:



Scenario	Step Count	Tool-Call Count	Stop Reason
Search trial	2	1	normal_completion
Trial details	2	1	normal_completion
Organization summary	2	1	normal_completion
Safety rule scenario	1	0	normal_completion

All four scenarios completed successfully using the local qwen3:8b model. The Search Trial scenario completed in two steps with one tool call. The Trial Details and Organization

Summary scenarios also completed in two steps with one tool call each. The Safety Rule scenario completed in one step without invoking a tool because the model recognized that the unrestricted wildcard request was not allowed and returned a safe response directly.

5.5 Reflection:

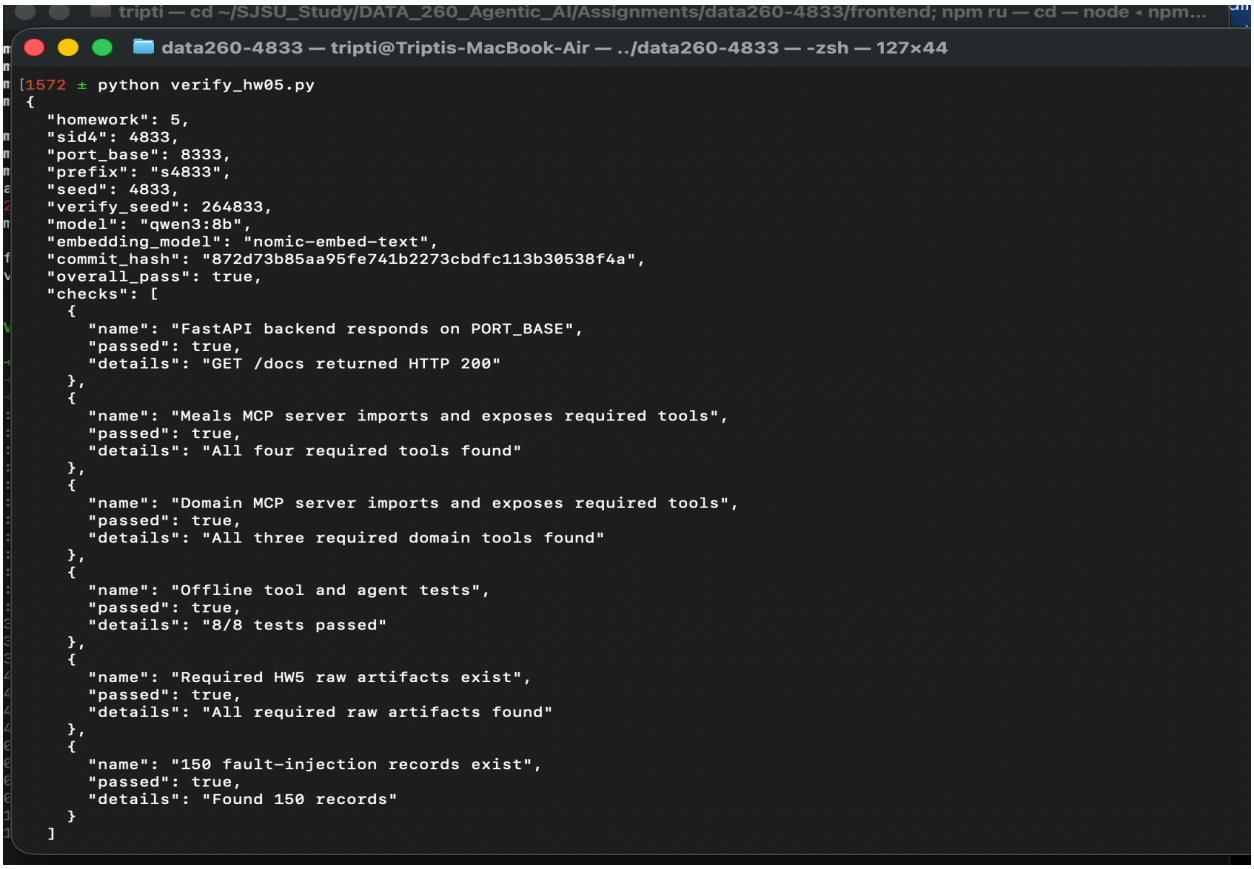
For this reflection, I selected the Search Trial scenario from agent_runs.jsonl. The user asked the agent to find clinical trials related to “Cardio.” In the first step, the agent interpreted the request and decided that it needed information from the domain data rather than answering from its own knowledge. It therefore called the search_trials tool using Cardio as the search query. The tool returned one matching clinical trial, the Cardiovascular Health Study, along with its trial code, participant capacity, and organization ID.

After receiving the tool result, the agent had enough information to answer the original request. In the second step, it generated a concise final response summarizing the matching clinical trial instead of making another unnecessary tool call. The complete run therefore used two agent steps and one tool call.

The stop reason was normal_completion because the model determined that the user’s question had been answered successfully. This run helped me understand why an agent should not call tools unnecessarily. The agent first retrieved grounded information from the domain tool and then stopped as soon as it had enough information to respond. The max_steps limit still acts as a safeguard if the model continues requesting tools, while the JSONL logging makes each decision, tool call, and final stopping condition traceable.

Part 6 – Verification

I ran the HW5 self-check script after completing the implementation. The verification confirmed that the FastAPI backend responded on the assigned port, both MCP servers exposed the required tools, all 8 offline tests passed, all required raw artifacts were present, and all 150 fault-injection records were successfully generated. The final verification result was overall_pass: true.



Part 7 – AI Use Disclosure

I used an AI assistant for assignment guidance, debugging, and starter code structure, while I personally executed and verified all application, MCP, experiment, and agent runs. One AI-generated suggestion initially used an MCP import that did not match my installed MCP version. I identified the problem through the terminal traceback and MCP Inspector failure, then updated the implementation and verified that all required MCP tools connected and executed successfully.